

# **Computer Vision**

**2026. 2. 2**

**3C AI Campus**  
**박 준 오**

### 3. OPENCV

---



# 3. OPENCV

---

## ■ OpenCV 핵심 용어

### ① 이미지 & 데이터 구조

|                      |                                         |
|----------------------|-----------------------------------------|
| <b>Pixel</b>         | 이미지의 최소 단위, 밝기 또는 색상 값을 가짐              |
| <b>Resolution</b>    | 이미지의 크기 (Height × Width)                |
| <b>Channel</b>       | 색상 정보를 구성하는 축 (B, G, R)                 |
| <b>BGR</b>           | 기본 색상 순서 (RGB 아님 ⚠)                     |
| <b>OpenCV</b>        |                                         |
| <b>RGB</b>           | 일반적인 색 표현 방식 (Red, Green, Blue)         |
| <b>Grayscale</b>     | 색을 제거하고 밝기만 남긴 단일 채널 이미지                |
| <b>HSV색</b>          | (Hue), 채도(Saturation), 명도(Value) 기반 색공간 |
| <b>Color Space</b>   | 색을 표현하는 좌표계 개념                          |
| <b>numpy.ndarray</b> | OpenCV 이미지의 실제 데이터 타입                   |
| <b>Dtype</b>         | 이미지 픽셀 데이터의 자료형                         |

# 3. OPENCV

---

## ② 전처리 & 필터링 (Low-level Vision)

**Thresholding** 픽셀 값을 기준으로 이진화하는 과정  
이미지의 픽셀 값이 어떤 기준값(임계값)을 기준으로 나누는 과정  
즉, 회색 이미지를 흑백으로 바꾸는 가장 기본적인 방법.

**Binary Image** Thresholding을 거친 결과 이미지

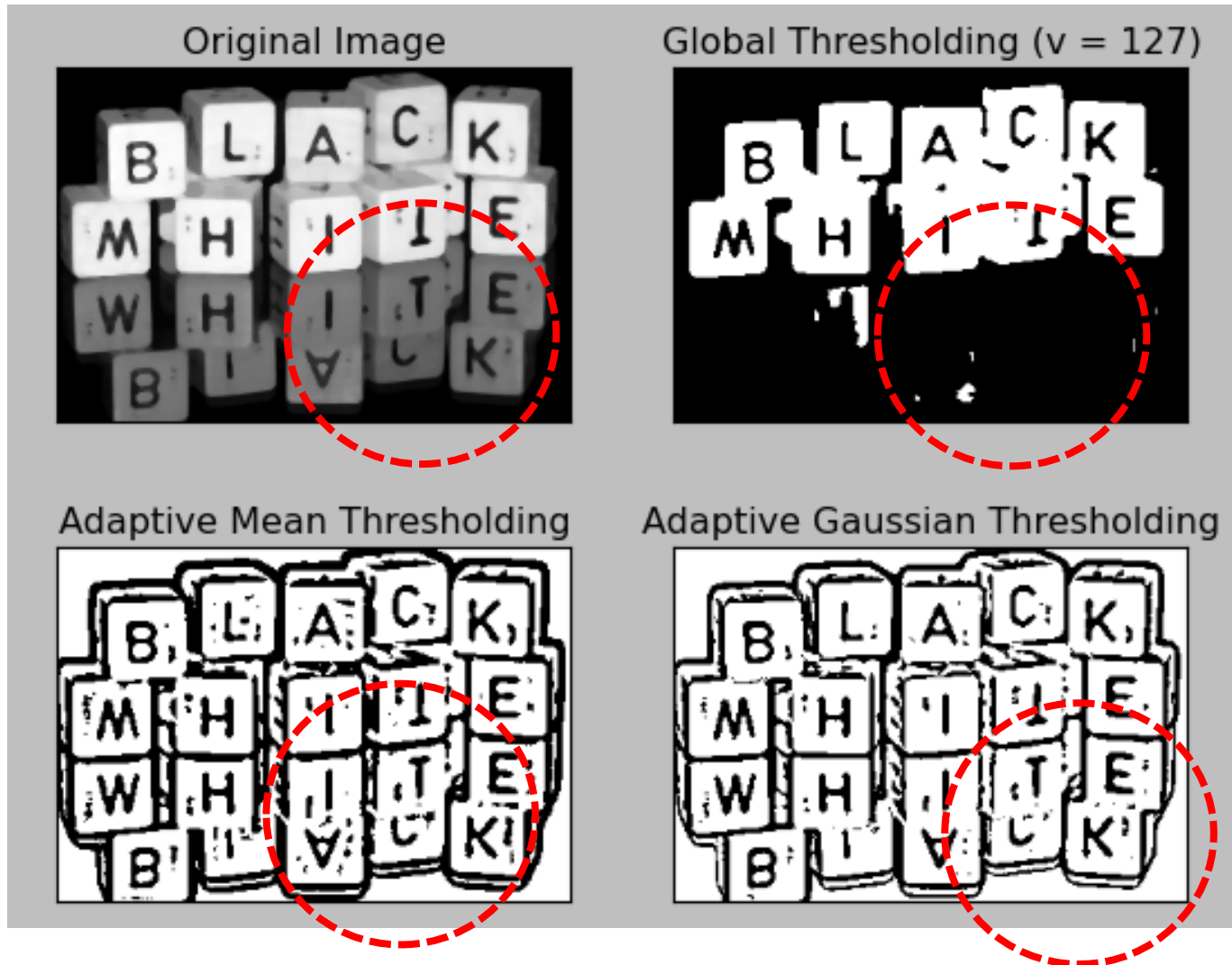
**Global Threshold** 이미지 전체에 하나의 동일한 임계값을 적용하는 방식  
고정 임계값을 사용하는 이진화

**Adaptive Threshold** 지역별로 다른 임계값을 적용하는 이진화  
이미지를 작은 영역들로 나누고,  
각 영역마다 다른 임계값을 자동으로 계산해서 이진화

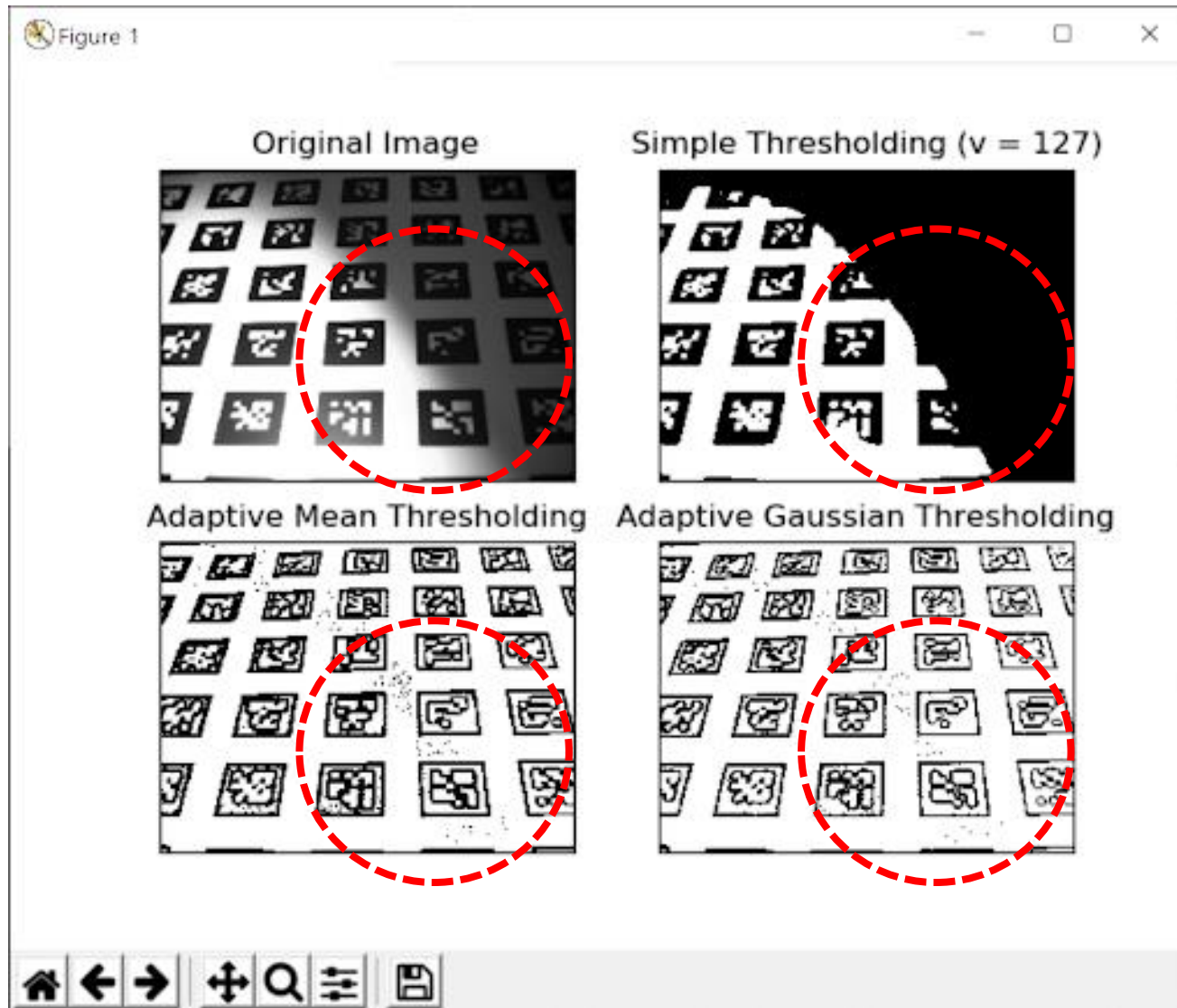


## 3. OPENCV

---



## 3. OPENCV



# 3. OPENCV

---

## ② 전처리 & 필터링 (Low-level Vision)

|                      |                           |
|----------------------|---------------------------|
| <b>Kernel</b>        | 필터 연산에 사용되는 작은 행렬         |
| <b>Convolution</b>   | 커널을 이미지에 슬라이딩하며 계산하는 연산   |
| <b>Gaussian Blur</b> | 가우시안 분포 기반의 블러 필터         |
| <b>Median Blur</b>   | 중앙값으로 노이즈를 제거하는 필터        |
| <b>Noise</b>         | 이미지에 포함된 원치 않는 픽셀 변화      |
| <b>Smoothing</b>     | 노이즈 제거를 위한 전체적인 이미지 완화 처리 |

# 3. OPENCV

---

## ③ Edge & 형태 분석 (Mid-level Vision)

### **Canny Edge**

가장 널리 쓰이는 엣지 검출 알고리즘

### **Contour**

객체의 외곽선을 이루는 점들의 집합  
Contour (프랑스어) = 윤곽, 등고선

### **findContours**

이진 이미지에서 contour를 추출하는 함수

## 3. OPENCV

---

Original Image



Canny Edge Detector



Contour Drawing



# 3. OPENCV

---

## ③ Edge & 형태 분석 (Mid-level Vision)

**Edge Detection**                      밝기 변화가 큰 경계를 검출하는 과정

**Gradient**                              픽셀 밝기 변화의 크기와 방향

**Bounding Box**                        객체를 감싸는 최소 사각형

**Areacontour**                          내부의 면적

**Perimeter (Arc Length)**            contour의 둘레 길이

### 3. OPENCV

---

④ 좌표계 & 기하 변환 (공간 이해)

**Coordinate System (좌표계)**, 이미지 상의 위치 표현 체계 (좌상단이 원점)

**ROI (Region of Interest)** 관심 영역만 잘라서 처리하는 기법

**Affine Transform** 영상 처리에서 최소한의 특성을 유지하며 변환하는 과정  
이동하는 변환. 단, 직선은 직선으로, 평행선은 평행선으로 유지

**Perspective Transform(원근 변환)** 카메라로 본 것처럼 가까운 것은 크게,  
먼 것은 작게 보이도록 바꾸는 변환. 3차원 느낌을 2차원에 반영

## 3. OPENCV

---

### ④ 좌표계 & 기하 변환 (공간 이해)

#### **warpAffine**

이미지를 이동·회전·확대·기울이기 하는 함수

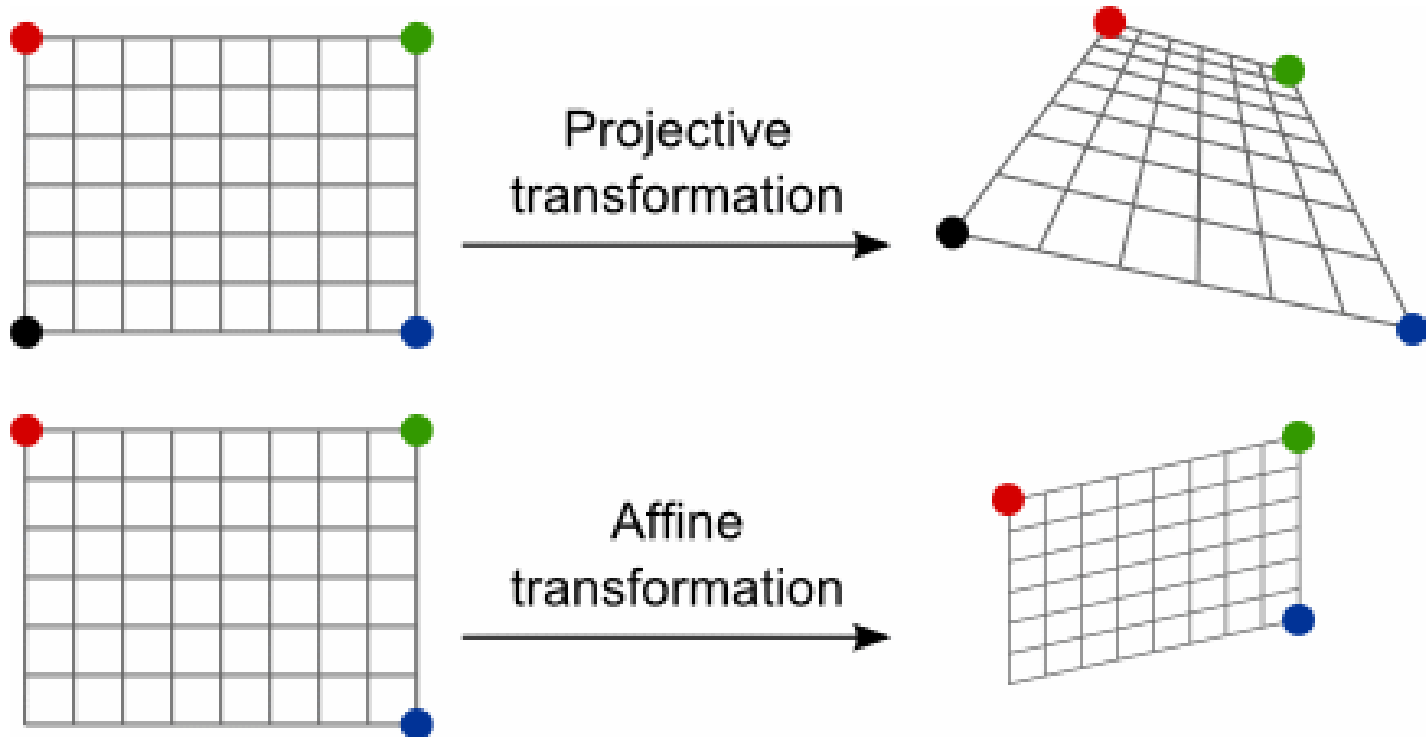
#### **warpPerspective**

이미지를 카메라 시점처럼 변형하는 함수  
가까운 것은 크게, 먼 것은 작게 표현합니다.



### 3. OPENCV

---



### 3. OPENCV

---

#### ⑤ Feature 기반 비전 & 딥러닝 연결

|                           |                         |
|---------------------------|-------------------------|
| <b>Feature</b>            | 이미지에서 의미 있는 시각적 특징      |
| <b>Feature Matching</b>   | 두 이미지 간 특징점을 대응시키는 과정   |
| <b>Preprocessing</b>      | 모델 입력 전에 수행하는 이미지 정리 과정 |
| <b>Normalization</b>      | 픽셀 값 범위를 조정하는 과정        |
| <b>Resize</b>             | 입력 크기를 통일하기 위한 스케일 조정   |
| <b>Annotation</b>         | 객체 위치 정보를 라벨로 표시한 데이터   |
| <b>Bounding Box Label</b> | 객체 탐지를 위한 좌표 기반 라벨      |

# 3. OPENCV

---

## OpenCV 핵심 명령어 50선

### ① 입출력 & 기본 조작

|                                |                      |
|--------------------------------|----------------------|
| <b>cv2.imread(path)</b>        | 이미지 파일을 NumPy 배열로 읽기 |
| <b>cv2.imshow(name, img)</b>   | 이미지를 윈도우에 출력         |
| <b>cv2.waitKey(ms)</b>         | 키 입력 대기 (0이면 무한 대기)  |
| <b>cv2.destroyAllWindows()</b> | 모든 OpenCV 창 닫기       |
| <b>img.shape</b>               | 이미지 크기 (H, W, C) 확인  |
| <b>img.dtype</b>               | 픽셀 데이터 타입 확인         |
| <b>img.copy()</b>              | 원본 보존용 복사본 생성        |

### 3. OPENCV

---

#### ② 색공간 & 채널 처리

**cv2.cvtColor(img, flag)**

**cv2.COLOR\_BGR2GRAY**

**cv2.COLOR\_BGR2HSV**

**cv2.split(img)**

**cv2.merge(channels)**

**cv2.inRange(img, lower, upper)**

색공간 변환 (BGR↔GRAY↔HSV)

BGR → Grayscale 변환 플래그

BGR → HSV 변환 플래그

채널 분리 (B, G, R)

분리된 채널 다시 합치기

특정 값 범위를 마스크로 생성

## 3. OPENCV

---

### ③ 이진화 & 필터링

**cv2.threshold(src, t, max, type)**

**cv2.adaptiveThreshold(...)**

**cv2.GaussianBlur(img, ksize, sigma)**

**cv2.medianBlur(img, ksize)**

**cv2.blur(img, ksize)**

**cv2.filter2D(img, depth, kernel)**

글로벌 임계값 이진화

지역 기반 이진화

가우시안 블러 (노이즈 제거)

중앙값 블러

단순 평균 블러

커스텀 커널 필터 적용

### 3. OPENCV

---

#### ④ Edge & 형태 처리

**cv2.Canny(img, t1, t2)**

엣지 검출 (Canny)

**cv2.findContours(img, mode, method)**이진 이미지에서 윤곽선 검출

**cv2.drawContours(img, contours, idx, color)**contour 시각화

**cv2.boundingRect(contour)**

contour의 최소 사각형 좌표

**cv2.contourArea(contour)**

contour 면적 계산

**cv2.arcLength(contour, closed)**

contour 둘레 길이 계산

### 3. OPENCV

---

#### ⑤ 형태학 연산

**cv2.erode(img, kernel)**

침식 (객체 줄이기)

**cv2.dilate(img, kernel)**

팽창 (객체 키우기)

**cv2.morphologyEx(img, op, kernel)** Opening / Closing 등 복합 연산

**cv2.MORPH\_OPEN**

침식 → 팽창 (노이즈 제거)

**cv2.MORPH\_CLOSE**

팽창 → 침식 (구멍 메우기)

### 3. OPENCV

---

#### ⑥ 좌표계 & 기하 변환

**cv2.resize(img, dsize)**

이미지 크기 변경

**cv2.getRotationMatrix2D(center, angle, scale)**

회전 행렬 생성

**cv2.warpAffine(img, M, dsize)**

Affine 변환 적용

**cv2.getPerspectiveTransform(src, dst)**

원근 변환 행렬 계산

**cv2.warpPerspective(img, M, dsize)**

원근 변환 적용

**cv2.rectangle(img, pt1, pt2, color)**

사각형 그리기 (Box)

**cv2.circle(img, center, r, color)**

원 그리기



### 3. OPENCV

---

#### ⑦ 딥러닝 연결

**cv2.normalize(src, dst, a, b, type)**

픽셀 값 정규화

**cv2.cvtColor(img, cv2.COLOR\_BGR2RGB)**

DL 프레임워크용 색 순서 변경

**img[y1:y2, x1:x2]**

ROI 추출 (NumPy 슬라이싱)

**cv2.imwrite(path, img)**

이미지 저장

**cv2.VideoCapture(0)**

카메라 입력 열기

**cap.read()**

비디오 프레임 한 장 읽기

## 3. OPENCV

---



Library ▾

Forum ▾

OpenCV University ▾

New  
Free Courses ▾

Services

Face Recognition

Contribute ▾

Resources ▾



### Claim Your FREE OpenCV Crash Course

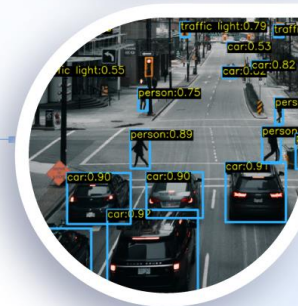
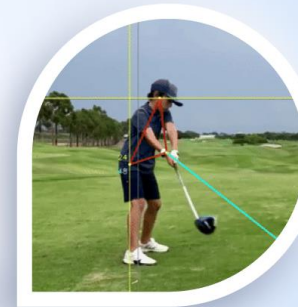
Dive into AI and Computer Vision, covering Image & Video Manipulation, Object and Face Detection, OpenCV Deep Learning Module and much more.

ENROLL NOW



### 3. OPENCV

# Computer Vision & Image Processing



# 3. OPENCV

---

## 이미지 처리(Image Processing)

사진/영상 자체를 더 보기 좋게 또는 전처리하기 위해 **픽셀 수준에서** 조작하는 기술

- 어두운 사진을 밝게 만들기,
- 노이즈 제거,
- 대비 조정,
- 가장자리 검출 등

이미지 자체를 바꾼다

주로 **수학적 알고리즘 / 픽셀 조작 중심**

# 3. OPENCV

---

## 컴퓨터 비전(Computer Vision)

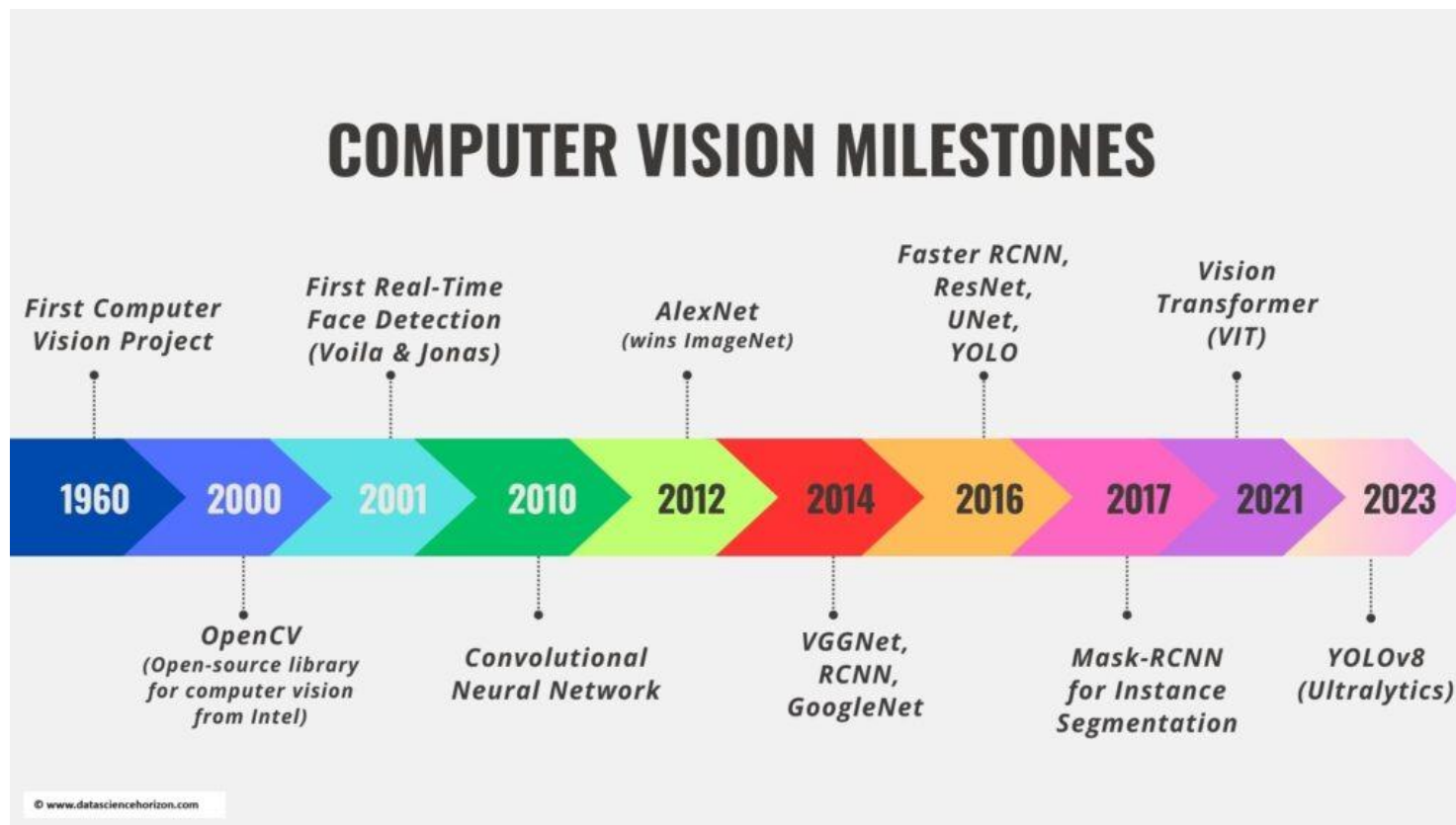
머신(컴퓨터)에게 **이미지/비디오의 의미를 이해하게** 하여

- 사진 속 사람/자동차/물건 인식,
- 객체 위치 파악,
- 장면 이해 등

이미지를 해석해서 의미 있는 정보를 얻는다

딥러닝 모델을 통해 의미 기반 예측/인식 수행

### 3. OPENCV 영상 처리 역사



## 3. OPENCV

### 1. 영상 처리의 태동 (1920년대)

영상 처리의 뿌리는 1920년대로 거슬러 올라갑니다.

런던과 뉴욕을 잇는 **해저 케이블**로 신문사가 사진을 전송하며 디지털 영상의 시작이 되었습니다.

초기 디지털 이미지 전송/복원 기술의 출발점.



### 3. OPENCV

---

## 2. 디지털 컴퓨터와 최초의 디지털 이미지 (1950~1960년대)

### ✓ 디지털 컴퓨터의 등장

1940년대 이후 디지털 컴퓨터 개념이 등장하면서 영상 처리를 위한 기초가 생겼습니다.

### ✓ 최초의 디지털 이미지

1957년, Russell A. Kirsch가 세계 최초로 **디지털 이미지 스캐너**를 만들어 디지털 사진을 컴퓨터에 저장했습니다.→ 이 작업은 디지털 영상 처리 연구의 출발점으로 평가됩니다.

### ✓ 컴퓨터 비전의 초기 실험

1950~60년대에는 컴퓨터가 **이미지에서 형태/선 속성을 인식하려는 연구**들이 시작되었습니다.

예: **가장자리를 검출하고 단순한 도형을 분류**하는 알고리즘 연구.



## 3. OPENCV

### 3. 본격적인 영상 처리 / 우주 탐사 (1960~1970년대)

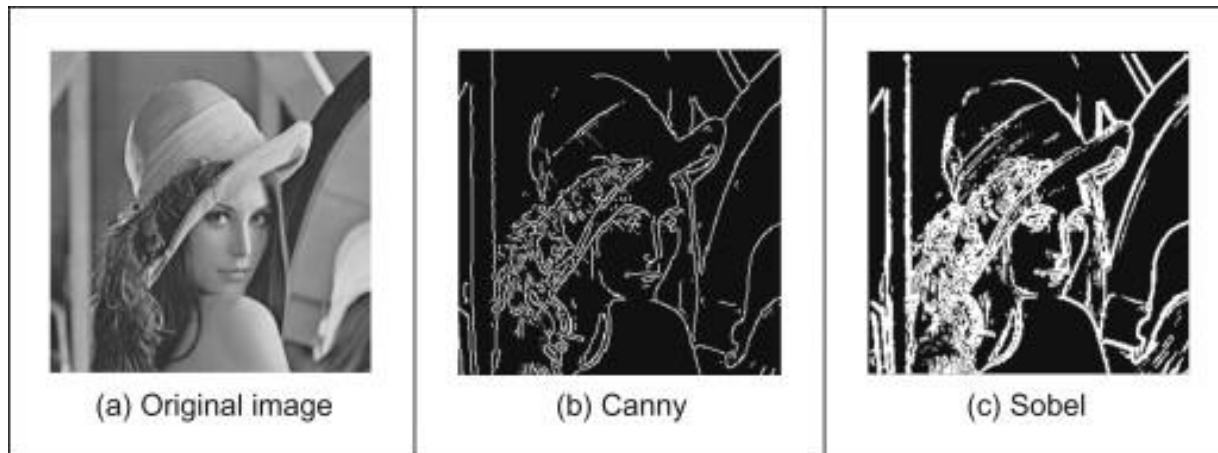
#### 우주 탐사와 영상 처리

1960년대 후반, 우주 탐사선이 보내온 사진을 보정/복원하는 기술이 필요해 졌습니다.  
예: 달 탐사선이 보내온 훼손된 영상들을 디지털 방식으로 정리함으로써 이미지 처리의 중요성이 부각됐습니다.

알고리즘 개발의 시작

**엣지 검출**과 같은 기초 영상 처리 알고리즘이 개발된 시기.

**Sobel/Canny edge detector** 같은 경계 검출 기술.



### 3. OPENCV

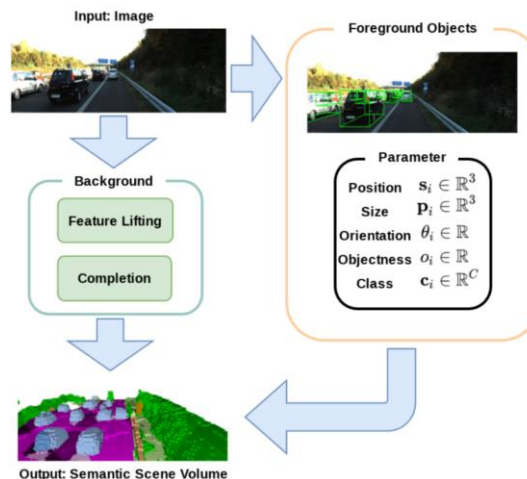
#### 4. 컴퓨터 비전의 출현 (1970~1980년대)

컴퓨터 비전은 단순히 이미지 처리(픽셀 조작)를 넘어서  
기계가 **이미지에서 의미 있는 정보를 이해하도록** 하는 연구

##### 객체의 3D 구조 추론

컴퓨터가 2D 이미지에서 **3D 정보를 해석하려는 연구.**

● **패턴 인식 / 윤곽 분석** 등이 발전해 **이미지 이해의 깊이가 확장.**



# 3. OPENCV

---

## 5. 1990년대와 디지털 혁명

- **디지털 영상/카메라 보급**

디지털 카메라와 이미지 센서(CCD/CMOS)의 발달로 사진·영상 데이터가 폭발적으로 증가.

- **인터넷 시대와 컴퓨터 비전 확장**

인터넷 보급으로 영상 데이터가 가용해지고,  
실시간 얼굴 인식, 객체 검색, 이미지 분류 같은 기술이 연구·응용.

### 3. OPENCV

---

#### 6. 머신러닝·딥러닝 시대 (2000~2010년대 이후)

- CNN의 부상

**Convolutional Neural Networks (CNN)**은 이미지 분류/인식 분야에 혁신  
딥러닝 기반 컴퓨터 비전의 표준

- YOLO와 실시간 객체 탐지

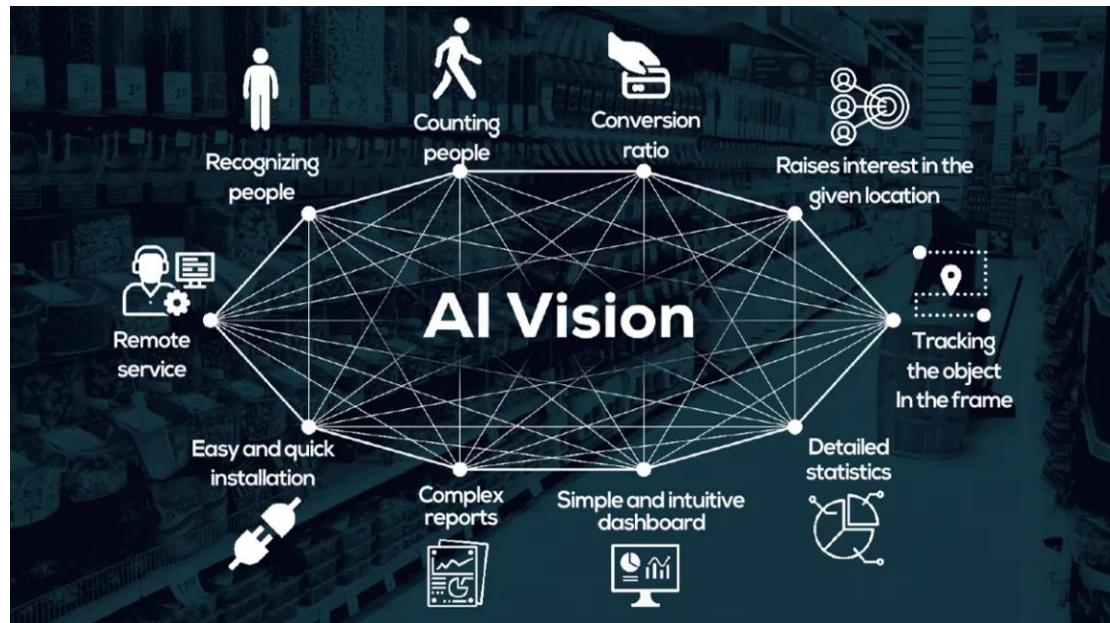
YOLO 계열 모델은 실시간 객체 탐지를 가능하게 했고,  
로봇/자율주행/영상 검수 같은 산업 적용을 촉진.

## 3. OPENCV

### 7. 현재와 미래

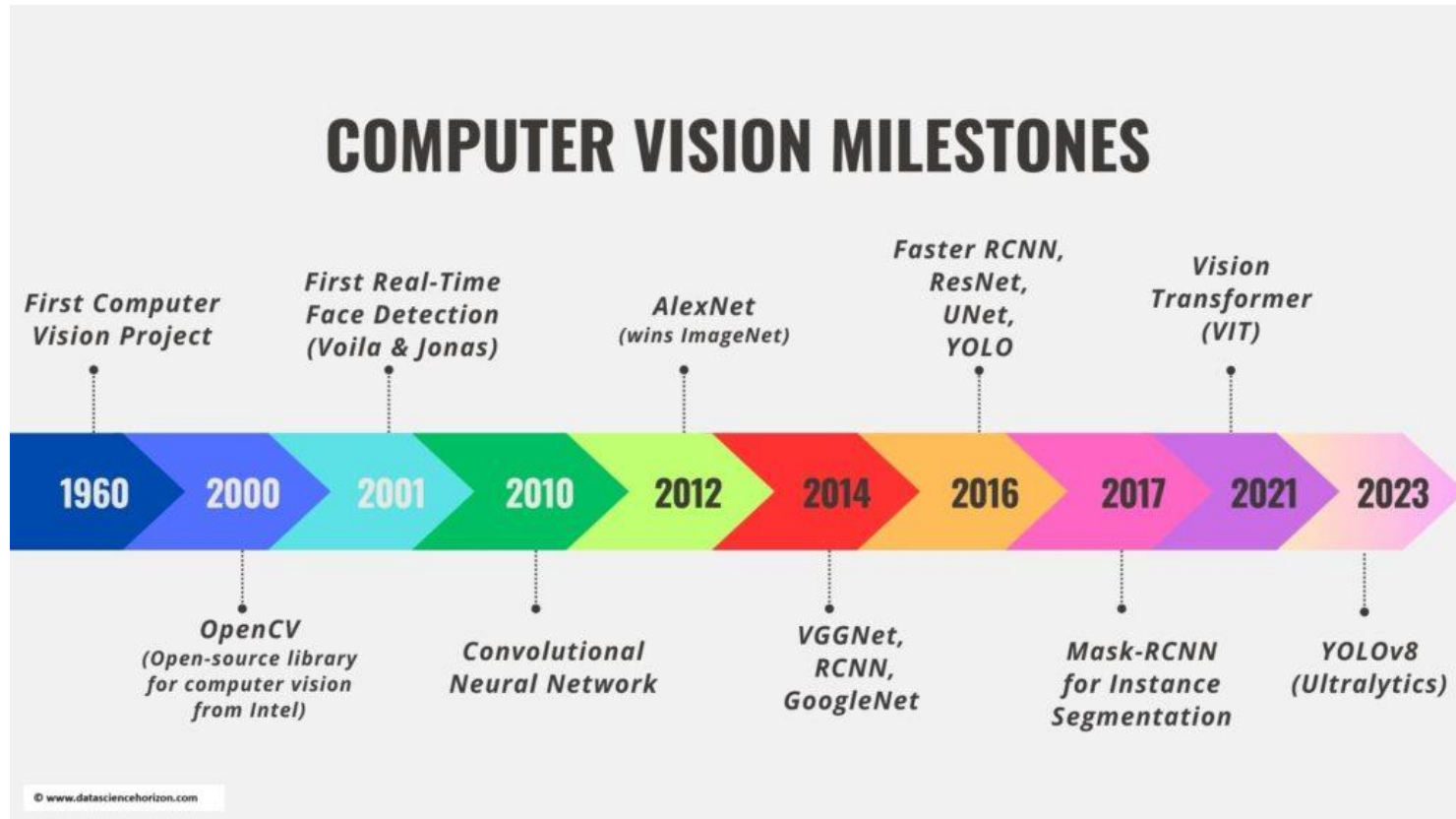
오늘날 컴퓨터 비전은 의료 진단, 자율주행, 제조/품질검사, 증강현실(AR), 소비자 앱(UIs) 등 다양한 산업에서 핵심 기술로 쓰이고 있습니다.

**기초 영상 처리 + 컴퓨터 비전 + 딥러닝**이 합쳐져  
현대의 강력한 **비전 AI 시스템**이 됩니다



### 3. OPENCV

---



### **3. OPENCV 영상 처리 응용 분야**

---

**영상 처리 응용 분야**

### 3. OPENCV 영상 처리 응용 분야

---

#### 1. 의료 분야

의료 영상(CT, MRI, PET, X-ray, 초음파 등)을 처리 및 분석해 질병 진단·추적·치료.

- CT/MRI 영상에서 종양 영역 자동 분할/검출
- 초음파 영상의 노이즈 제거 및 구조 해석
- PET 영상으로 기능적 변화 분석, (의료용 양전자 방출 단층촬영)
- AI가 특성 추출해 진단 보조 도구로 사용

실제 의료 분석에서는 딥러닝 기반 세분화/검출 모델과 전통 영상 처리 기술이 결합.



# 3. OPENCV

---

## 2. 방송·통신 분야

디지털 방송서 영상 품질 향상, 콘텐츠 처리, 방송 그래픽/광고 삽입 등에 사용

- **스포츠 방송에서 가상 광고(virtual advertising) 삽입**
- **화면 리마스터링 및 노이즈 제거**
- **실시간 영상 스트리밍에서 프레임 보정/압축 최적화**

# 3. OPENCV

---

## 3. 공장 자동화 / 산업용 응용

산업용 카메라로 제품 품질을 자동 검사하고, 불량을 즉시 찾아냅니다.

- **생산 라인에서 제품 표면 균열·오염 감지**
- **부품 치수/형태 검사**
- **로봇 팔의 대상 위치/방향 인식**

**머신 비전(machine vision)**

### 3. OPENCV

---

#### 4. 기상 및 지질 탐사 분야

기상/지질 데이터를 시각화하고, 다양한 자료를 영상으로 처리합니다.

- **위성 이미지로 구름 패턴/폭풍 경로 분석**
- **지형/지질 구조 모델링**
- **농업/환경 모니터링**

다중 스펙트럼 영상 처리(가시광선 + 적외선 등)로 정보 추출

## 3. OPENCV

---

### 5. 애니메이션·게임 분야

촬영 영상과 그래픽 기술을 조합해 현실감 있는 장면을 만들고, 시각 효과를 강화.

- **모션 캡처 데이터 처리**
- **프레임 합성/렌더링**
- **영상 기반 캐릭터 행동 인식**

실시간 그래픽/렌더링에서도 Computer Vision 알고리즘이

# 3. OPENCV

---

## 6. 자율주행 및 교통 시스템

주변 환경 인식, 차량/보행자 탐지, 차선 인식 등 자율주행 에 영상 처리·컴퓨터 비전이 사용

- 실시간 객체 탐지
- 거리/속도 계산
- 차선/신호 인식

자율 시스템은 영상 처리 + 센서 융합 + AI 판단으로

## 3. OPENCV

---

### 7. 보안/감시 분야

사람/사물의 움직임 탐지, 이상 행동 분석, 얼굴/번호판 인식 등 보안 관련 시스템

- **인원 수 세기 / 이상 행동 경고**
- **출입 통제 시스템**
- **CCTV 영상 자동 분석**

### **3. OPENCV 컴퓨터 비전 처리 단계**

---

## **컴퓨터 비전 처리 단계**

### 3. OPENCV 컴퓨터 비전 처리 단계

---

#### 1) 입력 및 전처리 (Preprocessing)

원시 이미지(또는 영상)를

→ 노이즈 제거

→ 밝기/색상 표준화

→ 왜곡 보정된 상태로 만드는 단계

왜 필요한가?

카메라/센서로 들어온 영상은

빛 조건이 다르고 노이즈가 있고 해상도가 일정하지 않을 수 있다.

이를 그대로 분석하면

✗ 잘못된 특징이 나오거나

✗ 모델이 잘못 판단할 수 있기 때문에 전처리가 필수

대표 기술

색 변환 / 그레이스케일 변환, 가우시안 블러링 등 노이즈 제거  
히스토그램 정규화(밝기 균등화)



# 3. OPENCV

---

## 2) 특징 추출 (Feature Extraction)

영상에서 중요한 구조.패턴만 뽑아내는 단계입니다.

원시 이미지 픽셀 전체가 아니라

**의미 있는 정보(에지, 코너, 질감, 형태 등)만 숫자로** 변환합니다.

왜 중요한가?

이미지는 수백만 개의 픽셀로 구성됩니다.

머신/AI가 전체 픽셀을 일일이 처리하면 매우 느리고, 불필요한 정보가 많습니다.

**핵심 정보를 요약/정리해야** 효율적으로 판단할 수 있습니다.

# 3. OPENCV

---

## 이미지의 대표적인 특징(Feature)

### ✓ 에지(Edge)

- 물체 경계/윤곽선을 의미
- 밝기 급변 부분 탐지 → 장애물/객체 외곽 파악 기본 툴

### ✓ 코너(Corner)

- 두 방향 에지가 만나는 점
- 객체/형태 인식 핵심 포인트

## 특징 추출의 역할

- ✓ 노이즈 많은 영상에서 신뢰할 수 있는 정보 뽑기
- ✓ 계산량 감소 → 무의미한 픽셀 정보 대신 의미 있는 벡터만 다룸
- ✓ 이후 모델/AI의 분류·인식 성능 향상

## 3. OPENCV

---

### 3) 고수준 처리 (High-Level Processing)

추출된 특징을 바탕으로 의미를 해석/판단하는 단계입니다.

- **분류(Classification)** → 사진 속 인물이 개/고양이인지 구분
- **객체 탐지(Object Detection)** → 사람/차량/상품 등 위치/클래스 식별
- **세그멘테이션(Segmentation)** → 이미지의 픽셀 단위로 객체별 영역 분리
- **행동/장면 분석** → 동영상에서 사람 행동 인식

### 3. OPENCV 이미지와 색

---

이미지와 색

### 3. OPENCV

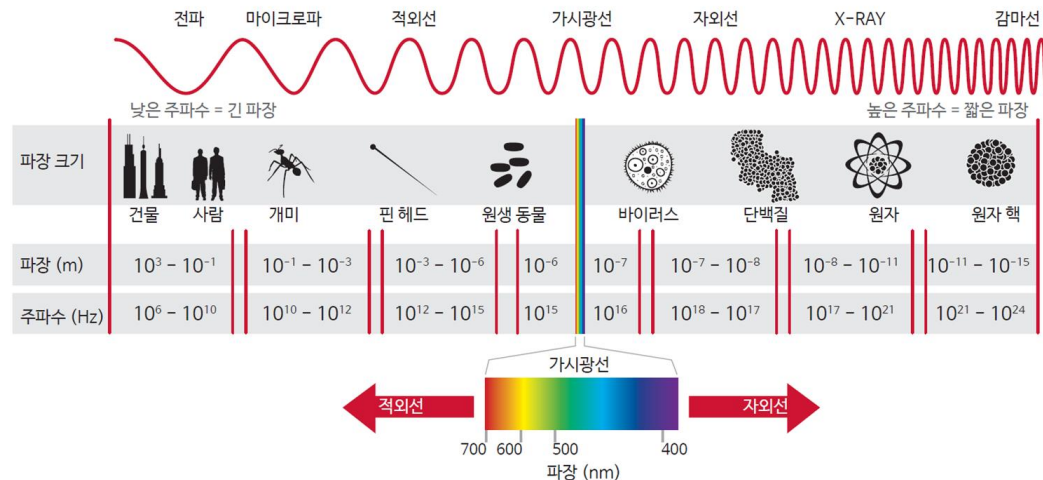
## 이미지와 색(Color)의 기초

색(빛)이란?

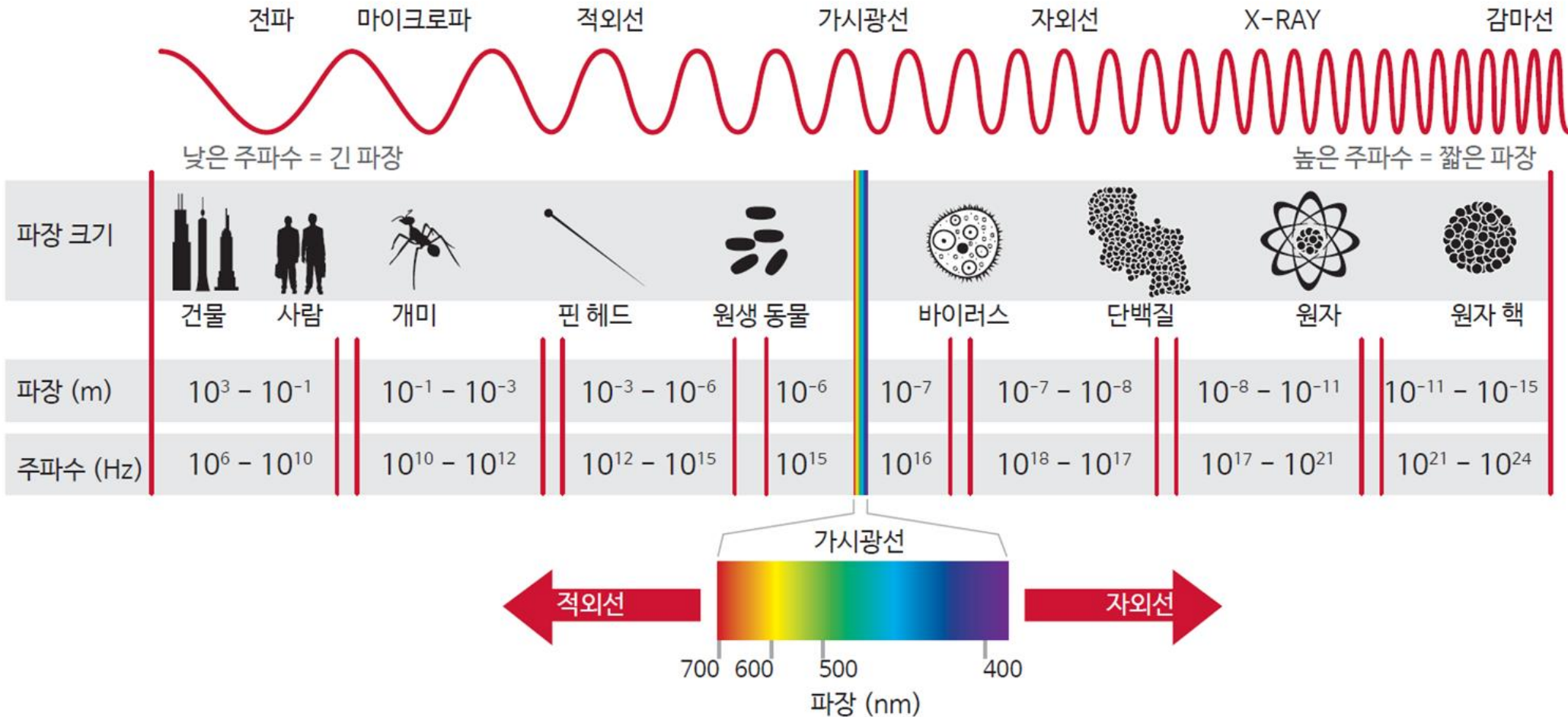
빛은 전자기파의 한 종류이고,

인간의 눈은 약 620 nm ~ 380 nm 범위의 파장을 인지해 색을 느낍니다.

이 범위가 **가시광선**입니다



### 3. OPENCV



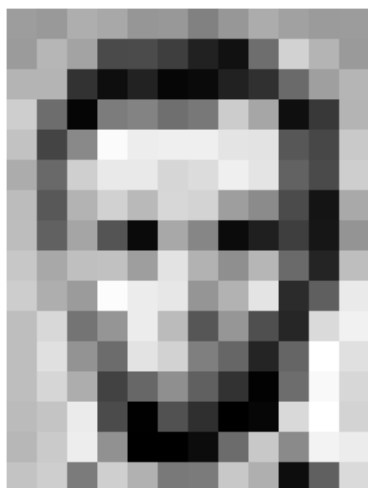
## 3. OPENCV

### 화소(Pixel, 畫素)란?

디지털 영상(이미지)을 구성하는 가장 작은 단위

영어 이름 pixel은 **picture + element**의 합성어

우리가 보는 이미지(사진/영상)는 수많은 작은 점(화소)으로 표현 가능



|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 131 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 237 | 239 | 239 | 228 | 227 | 87  | 71  | 201 |
| 172 | 106 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 85  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 35  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 95  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 131 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 237 | 239 | 239 | 228 | 227 | 87  | 71  | 201 |
| 172 | 106 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 85  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 35  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 95  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

# 3. OPENCV

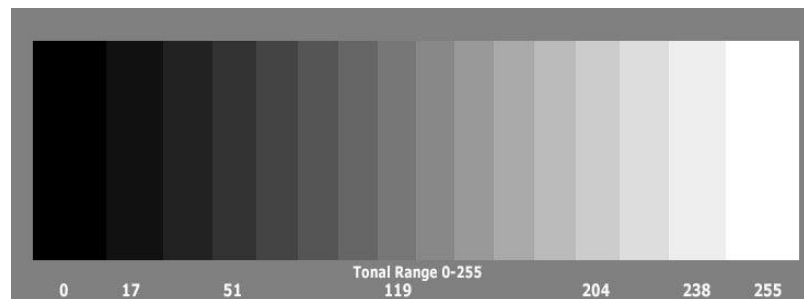
## 영상과 화소의 관계

**영상 = 화소들의 2차원 배열**

디지털 영상은 행(row)×열(column)로 구성된 2차원 배치(행렬).  
각 위치 하나가 하나의 화소입니다.

각 화소는 특정 빛의 세기/색 정보를 숫자로 나타냅니다.  
**숫자가 클수록 밝음, 작을수록 어두움을 의미합니다.**

예:0 → 완전 검은색  
255 → 완전 흰색 (흑백/그레이스케일 기준)  
(일반적으로 8비트 표현 기준)





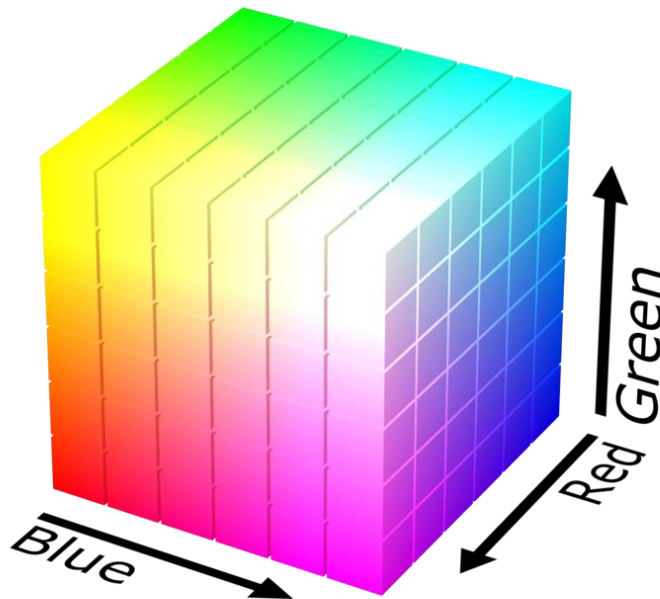
## 3. OPENCV

---

### 색공간(Color Space)이란?

**색을 수치로 표현하는 체계**이며,  
색을 3차원 좌표처럼 표현하는 수학적 구조.

색을 좌표처럼 표현할 수 있는 공간.



## 3. OPENCV

---

### 대표적인 색공간

#### 1) Grayscale (흑백) 색공간

- 색상 정보는 없고 명도 정보(밝기)만 있음
- 8바이트(8비트) 기준 0~255 사이 값으로 밝기만 표현
- 1채널(단일 값) 이미지입니다.
- 회색조 레벨 =  $2^n$  (여기서  $n$  = 비트 심도)



8-bit:  $2^8 = 256$  levels

10-bit:  $2^{10} = 1024$  levels

16-bit:  $2^{16} = 65,536$  levels

# 3. OPENCV

## 그레이스케일 이미지 (Grayscale)

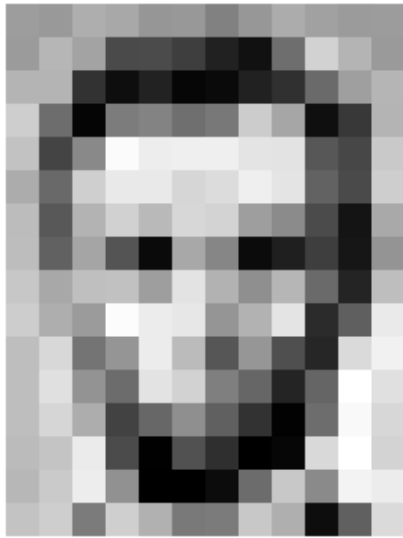
수학적 표현형태:

2차원 배열크기:  $H \times W$

각 원소 = 밝기(intensity)값이 클수록 밝음, 작을수록 어두움

### 컴퓨터 비전은 그레이?

- 데이터 차원 감소 (3채널  $\rightarrow$  1채널) 연산량 감소
- 조명 변화에 상대적으로 안정
- 경계/형태가 더 명확

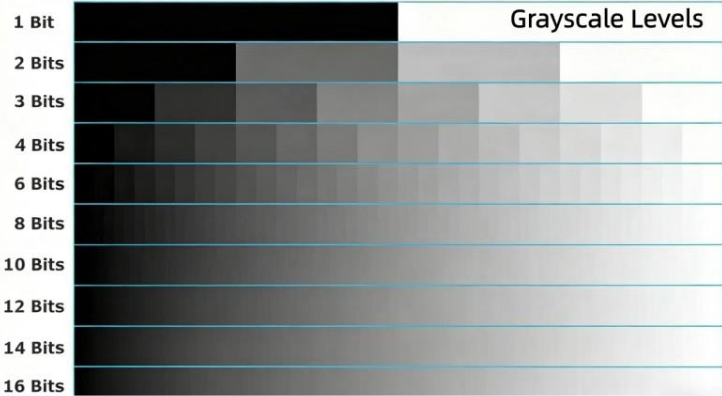


|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 191 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 257 | 299 | 299 | 228 | 227 | 87  | 71  | 201 |
| 172 | 105 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 86  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 36  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 96  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 191 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 257 | 299 | 299 | 228 | 227 | 87  | 71  | 201 |
| 172 | 105 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 86  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 36  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 96  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

### 3. OPENCV

| 비트 깊이  | 그레이스케일 레벨 | 설명/LED 디스플레이에 미치는 영향             |
|--------|-----------|----------------------------------|
| 1-bit  | 2         | 흑백만 있고 음영은 없습니다. 기본 이진 표시입니다.    |
| 2-bit  | 4         | 매우 제한된 색조, 간단한 그라데이션만 가능합니다.     |
| 4-bit  | 16        | 낮은 회색조, 이미지 세부 정보가 최소화되었습니다.     |
| 8-bit  | 256       | 표준 회색조, 부드러운 전환, 널리 사용됨.         |
| 10-bit | 1,024     | 음영이 더 정확해지고 이미지 디테일이 더 좋아졌습니다.   |
| 12-bit | 4,096     | 높은 회색조, 부드러운 그라데이션, 풍부한 색상 깊이.   |
| 14-bit | 16,384    | 매우 높은 회색조, 매우 부드러운 이미지와 정밀한 디테일. |
| 16-bit | 65,536    | 매우 높은 회색조, 거의 사진과 같은 음영 처리.      |



# 3. OPENCV

---

## 2) RGB 색공간

가장 널리 쓰이는 기본 색공간

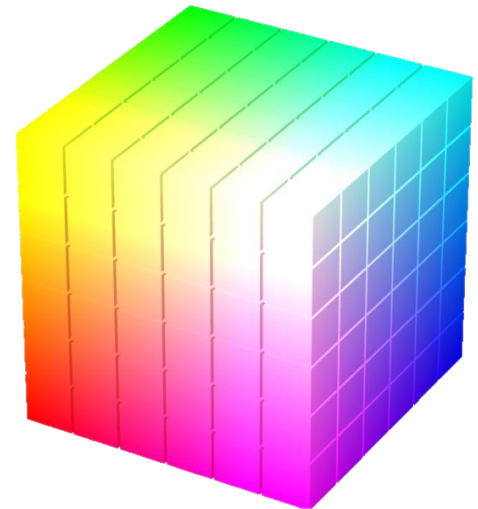
R, G, B는 빛의 물리적 삼원색

디스플레이 하드웨어(LED, 모니터)에 최적화, 기계 중심

📌 (255, 0, 0) → 빨강  
(0, 255, 0) → 초록  
(0, 0, 255) → 파랑

(0, 0, 0) → 검정  
(255, 255, 255) → 흰색

RGB 색공간은 빛의 가산혼합(Additive Mixing) 기반으로 색을 표현:  
**빛이 더해질수록 밝아지고 흰색에 가까워집니다.**



## 3. OPENCV

### 컬러 이미지 (BGR / RGB)

수학적 표현형태:

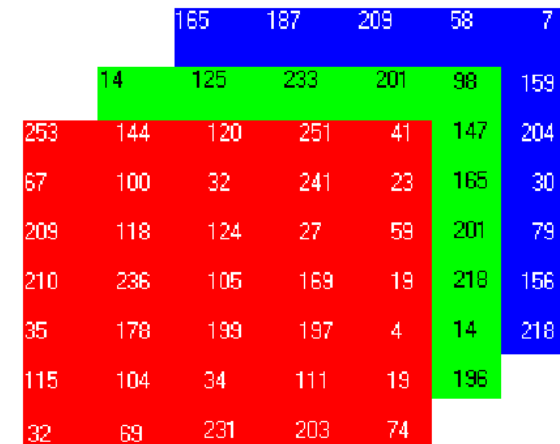
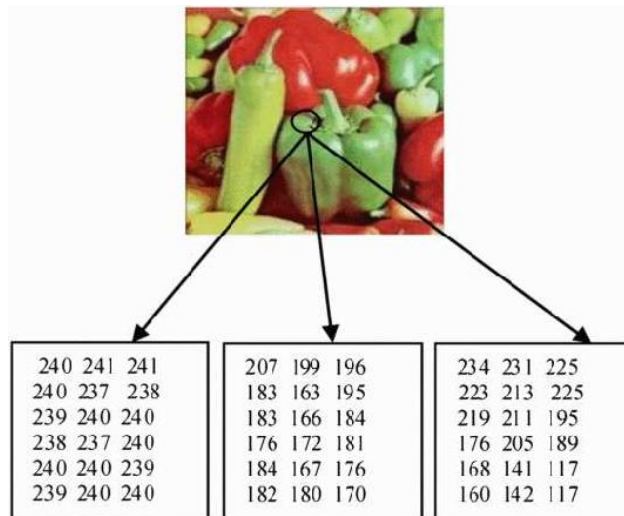
3차원 배열크기:  $H \times W \times 3$

각 픽셀은 3개의 숫자로 표현됨

RGB: (Red, Green, Blue)

픽셀 하나 =  $[R, G, B] = [255, 0, 0] \rightarrow$  빨강

**BGR: (Blue, Green, Red)  $\leftarrow$  OpenCV 기본**



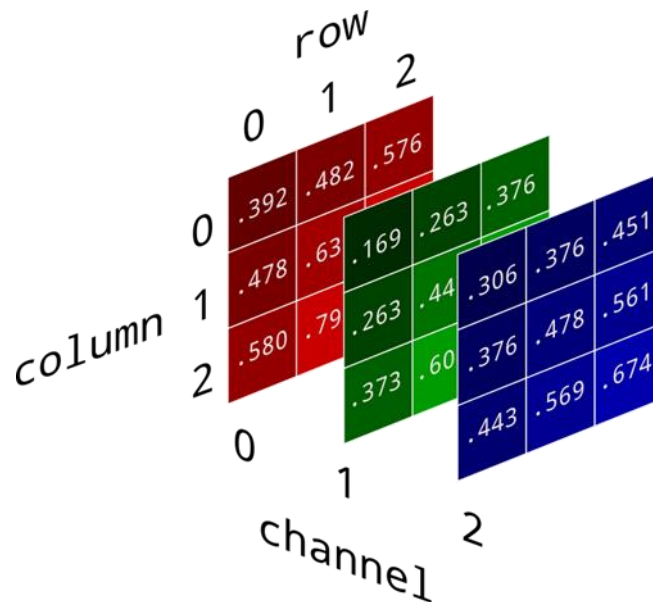
# 3. OPENCV

---

딥러닝에서의 표현자료형:  
float32값 범위: 0 ~ 1

- 수치 안정성
- 학습 최적화(gradient 안정)

`img_float = img_uint8 / 255.0`



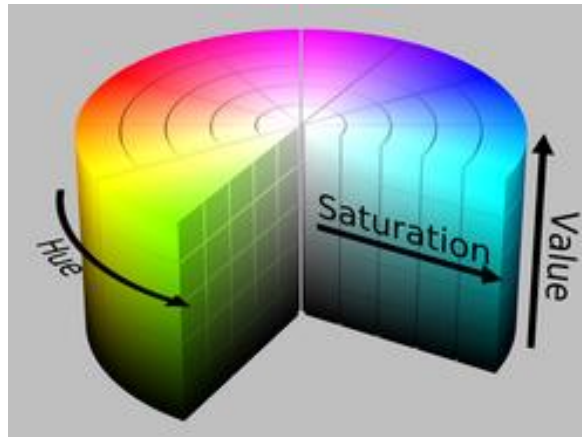
## 3. OPENCV

### 3) HSV 색공간

**사람 눈에 친숙한 방식으로 색을 표현.**

- H (Hue) 색상 (빨강→주황→노랑→초록→파랑→보라→ 빨강)
- S (Saturation) 채도 (색의 순수/선명함)
- V (Value) **명도 (밝기)**

색상/채도/명도를 분리해서 처리할 수 있기 때문에  
색 기반 이미지 분석(예: 피부톤 분리, 배경 제거)에 자주 쓰입니다.





# 3. OPENCV

## 컴퓨터 비전에서는 HSV를 선호할까?

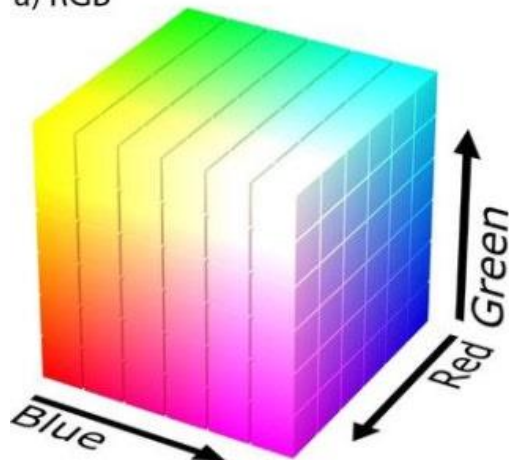
### RGB의 문제

- 조명 변화에 매우 민감
- 색과 밝기가 섞여 있음

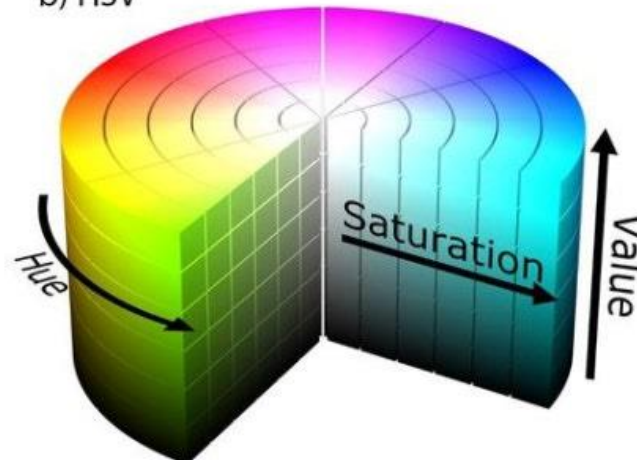
### HSV의 장점

- Hue는 조명 변화에 비교적 안정
- 색 검출 시 직관적

a) RGB



b) HSV



## 3. OPENCV

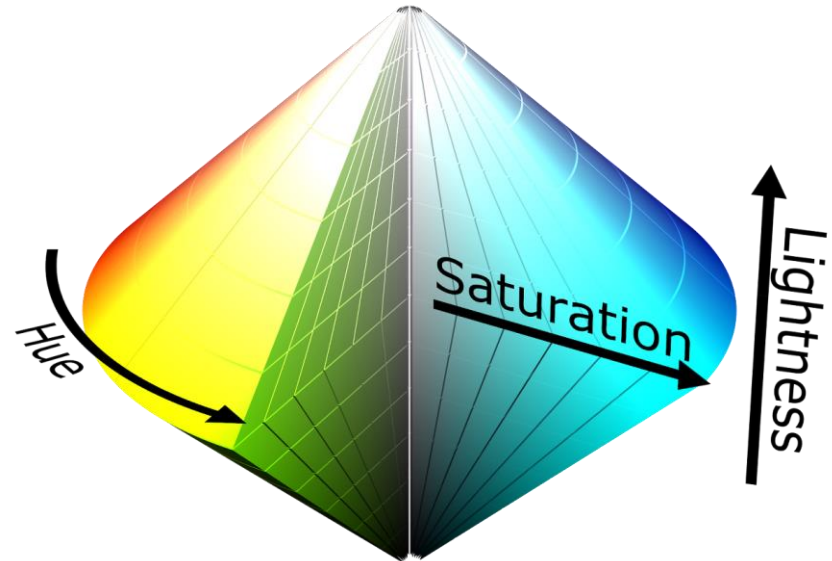
### 4) HSL

#### Hue(색상), Saturation(채도), Lightness(밝기)

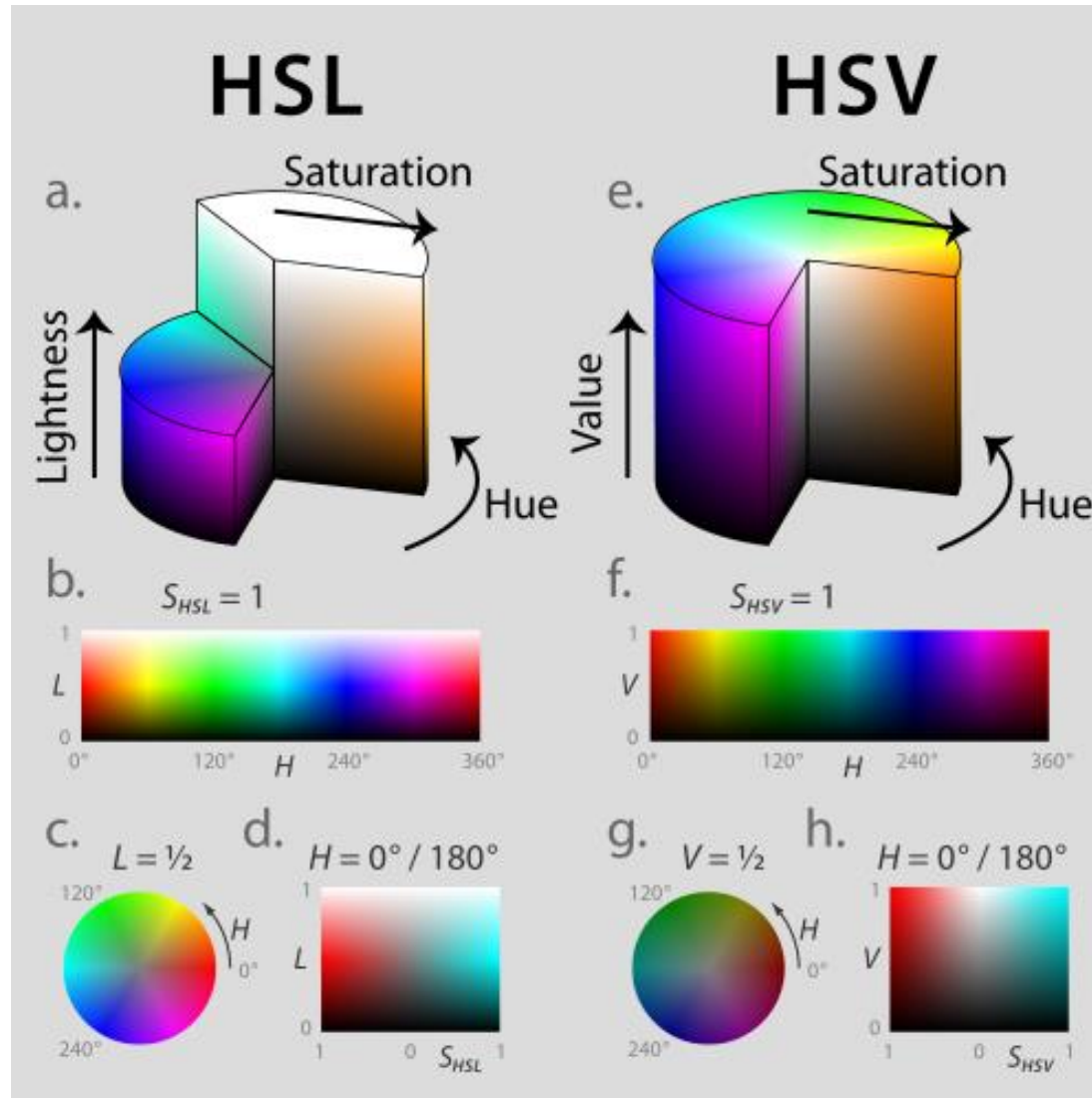
- Hue (색상):  $0^\circ$ 에서  $360^\circ$  사이의 각도로 색의 종류 (예:  $0^\circ$  빨강,  $120^\circ$  초록,  $240^\circ$  파랑)
- Saturation (채도): 색의 선명도. 0%는 회색(무채색), 100%는 가장 순수한 색
- Lightness (밝기): 흰색과 검은색의 배합 정도. 0%는 완전한 검은색, 100%는 완전한 흰색

HSV 색상 모델과 유사하지만 '값'을 사용하여 색상의 인지된 밝기를 나타내는 대신, '명도'를 사용하여 색상이 사람의 눈에 얼마나 밝거나 어둡게 보이는지를 나타냅니다.

"밝기를 올리면 결국 흰색이 된다"



### 3. OPENCV



# 3. OPENCV

| 항목         | RGB                           | HSV                                     | HSL                                     |
|------------|-------------------------------|-----------------------------------------|-----------------------------------------|
| 정의 방식      | 빨강/초록/파랑 <b>빛의 강도</b> 로 색 표현  | 색상/채도/ <b>값(밝기)</b> 으로 표현               | 색상/채도/ <b>명도(빛의 밝기 중심)</b> 표현           |
| 구조/차원      | 3차원 직교 좌표 (빛의 3원색)            | 원통형 좌표, v=밝기                            | 원통형 좌표, L=빛의 중심 밝기                      |
| 값 의미       | R/G/B 값 각각 0~255              | H: 색상(0~360°), S: 채도, <b>V: 밝기</b>      | H: 색상, S: 채도, <b>L: 명도</b>              |
| 밝기/명도 특징   | <b>밝기 개념이 직접 없음</b>           | v는 색의 <b>"강한 빛의 수준"</b> 강조              | <b>L은 "중간 톤 밝기"를 중심으로 좌/우로 흑/백까지 퍼짐</b> |
| 사람의 인지와 관련 | 모델/ <b>하드웨어 중심</b>            | <b>사람이 인지하는 색에 가까움</b>                  | <b>사람 중심 밝기 조작 최적화</b>                  |
| 표현 직관성     | 기술적(기계/디스플레이) 중심              | 중간 정도, 작업자 친화적                          | <b>디자이너/예술가 친화적</b>                     |
| 장점         | 디스플레이/렌더링에 최적,<br>하드웨어 가속 가능  | <b>색상과 밝기 분리</b> → 조명 변화 민감도 낮음         | <b>명도(L) 기준 색 조절 용이</b> → 색조/채도 독립 조작   |
| 단점         | <b>사람 인지 기준과 차이(색 직관 적음)</b>  | 명도/밝기 개념이 다소 혼합적                        | 일부 상황에서 채도 변동이 명도에 영향                   |
| 사용 용도 (예시) | 디지털 디스플레이,<br>기본 색 모델, 실시간 처리 | 이미지 색 분리/추출,<br>색 기반 객체 탐지, 조명 변화 있는 상황 | 그래픽/디자인 도구, 색 보정/선택,<br>예술 중심 작업        |

### 3. OPENCV 이미지 파일

---

#### 이미지 파일 형식과 특징

##### ■ BMP (Bitmap)

Windows 비트맵 형식으로 매우 약한 압축 이미지 저장 방식.  
모든 픽셀 데이터를 그대로 저장하므로 원본 화질을 그대로 유지합니다.

##### 장점

- 저장 구조가 간단하고 처리 속도가 빠름
- 원본과 동일한 이미지 품질을 그대로 보존함

##### 단점

- 파일 크기가 매우 큼 → 저장/전송 비효율적
- 웹 사용 및 모바일 환경에 부적합

**용도 :** 고품질 이미지 원본 저장  
개발/테스트용 원시 비트맵 처리

### 3. OPENCV

---

#### ■ **JPG/JPEG (Joint Photographic Experts Group)**

손실 압축(lossy compression)을 사용하는 대표적인 이미지 형식.

압축 과정에서 **인간 눈에 거의 느껴지지 않는 정보는 제거**되어 파일 용량이 줄어듦

#### **장점**

- 파일 크기가 작아 웹, 모바일, 전송 중심에 적합
- 사진 이미지 저장에 가장 널리 사용됨

#### **단점**

- 압축 시 원본 이미지 일부 정보가 손실됨
- 반복 저장 시 품질이 점점 떨어질 수 있음
- **투명도(알파 채널) 미지원**

#### **용도**

- 디지털 카메라 사진 저장
- **웹 이미지, 모바일 이미지**
- 고해상도 사진 공유

# 3. OPENCV

---

## ■ PNG (Portable Network Graphics)

**무손실 압축(lossless)**을 사용하는 현대적 이미지 형식입니다.

PNG는 **화질 손실 없이 용량 절감을 지원하며, 알파(투명도) 채널을 포함.**

### 특징

- PNG는 GIF보다 더 많은 색을 지원하고 투명도도 훨씬 정교하게 다룰 수 있음
- 반복 편집/저장이 가능하며 **품질 보존이 뛰어남**

### 단점

- JPG에 비해 파일 크기가 큰 편
- 애니메이션은 기본적으로 지원하지 않음

### 용도

- 웹그래픽(아이콘, 로고, 스크린샷)
- 투명 배경 이미지
- 이미지 분석·컴퓨터 비전
- 데이터 전처리

### **3. OPENCV 컴퓨터 비전과 OPENCV()**

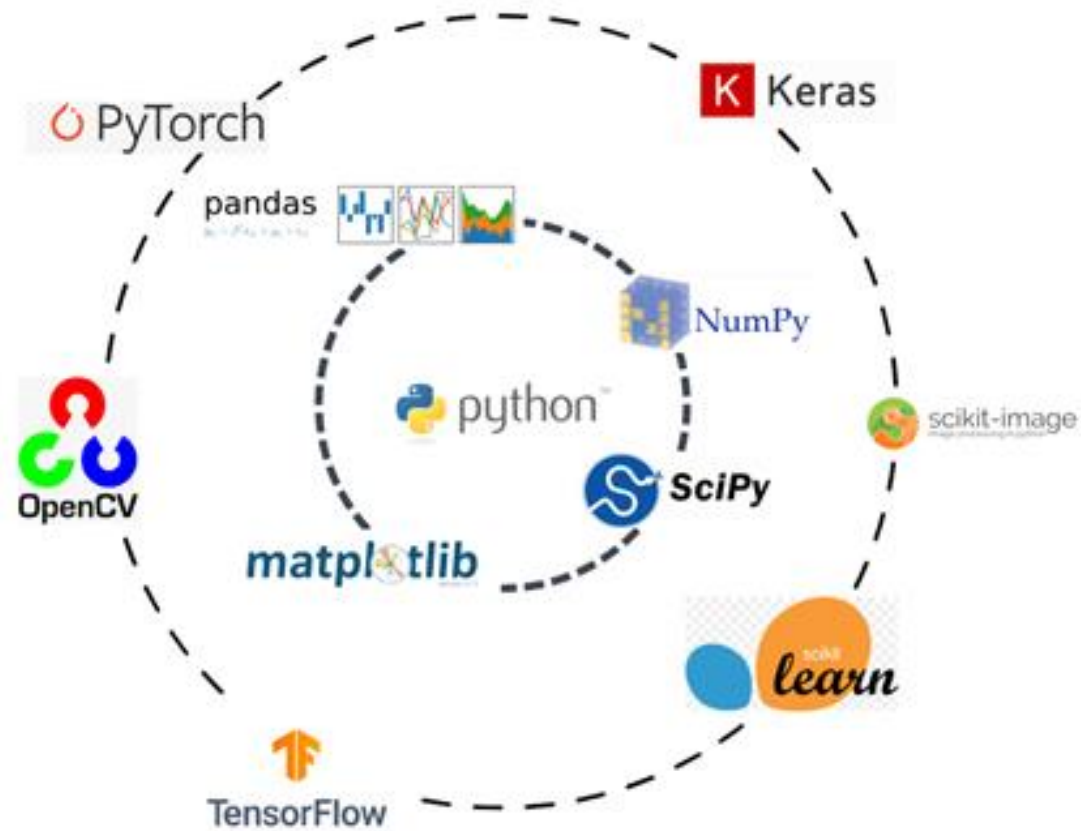
---

**컴퓨터 비전과 OPENCV()**



# 3. OPENCV

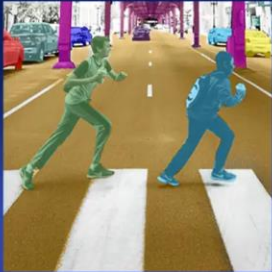
---



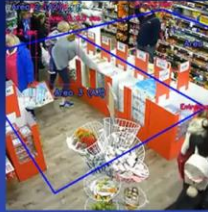
### 3. OPENCV



## Deep Learning for Computer Vision: Uses and Applications



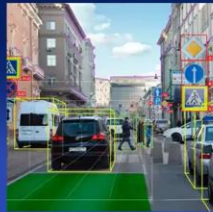
Semantic Segmentation



Localization



Image Synthesis



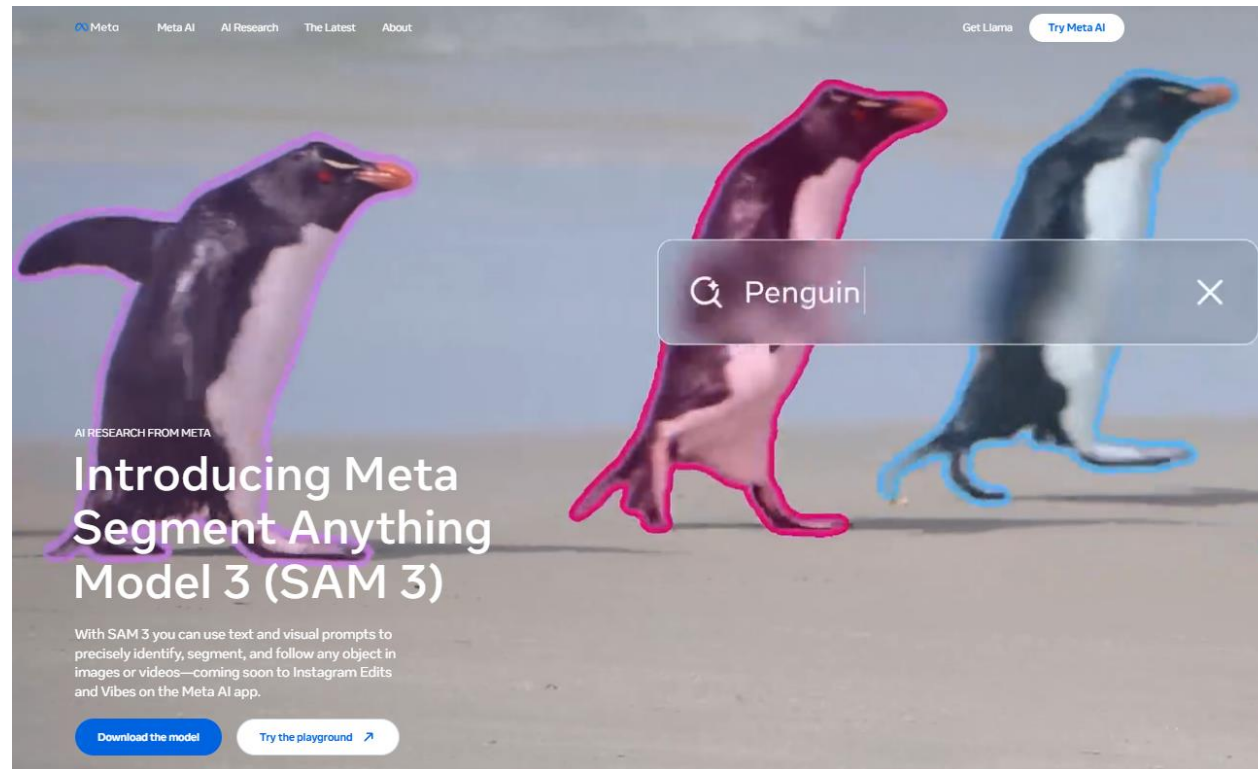
Object Detection



Image Super-Resolution

## 3. OPENCV

---



# 3. OPENCV

---

## 1차시 컴퓨터 비전 & OpenCV



1차시 목표

**“이미지는 숫자다”, OpenCV + NumPy**



주요 주제

- 컴퓨터 비전이란 무엇인가
- OpenCV의 역할과 한계
- 이미지의 수학적 표현



필수 이론 & 용어

- Pixel / Resolution
- `Image = numpy.ndarray`
- shape, dtype
- BGR vs RGB



필수 실습

- 이미지 불러오기 / 출력이미지 속성 확인
- 특정 픽셀 값 수정

# 3. OPENCV

---

## (1)컴퓨터 비전

**“이미지/영상”에서 의미(객체, 경계, 텍스트, 자세, 깊이 등) 를 추출하는 기술**  
**픽셀 값의 패턴을 수학적으로 다루는 것.**



# 3. OPENCV

---

## 컴퓨터 비전이 다루는 “의미”의 범위

컴퓨터 비전은 단순히 “보는 것”이 아니라, 보이는 것에서 의미를 추출하는 기술

대표적인 의미 정보

- **객체(Object):** 사람, 자동차, 동물, 제품 등
- **경계(Edge / Boundary):** 물체의 윤곽선
- **텍스트(Text):** 문자 인식(OCR)
- **자세(Pose):** 사람의 관절, 몸의 움직임
- **깊이(Depth):** 물체까지의 거리, 3D 구조
- **행동(Action):** 영상 속 행동 패턴

“픽셀” → “사물” → “상황”

### 3. OPENCV

픽셀 값의 특정 패턴

- **경계(edge)**→ 픽셀 값이 급격히 변하는 지점
- **객체(object)**→ 특정 색.형태.질감 패턴의 조합
- **얼굴(face)**→ 눈.코.입의 상대적 위치 패턴

**패턴을 인식 !!**

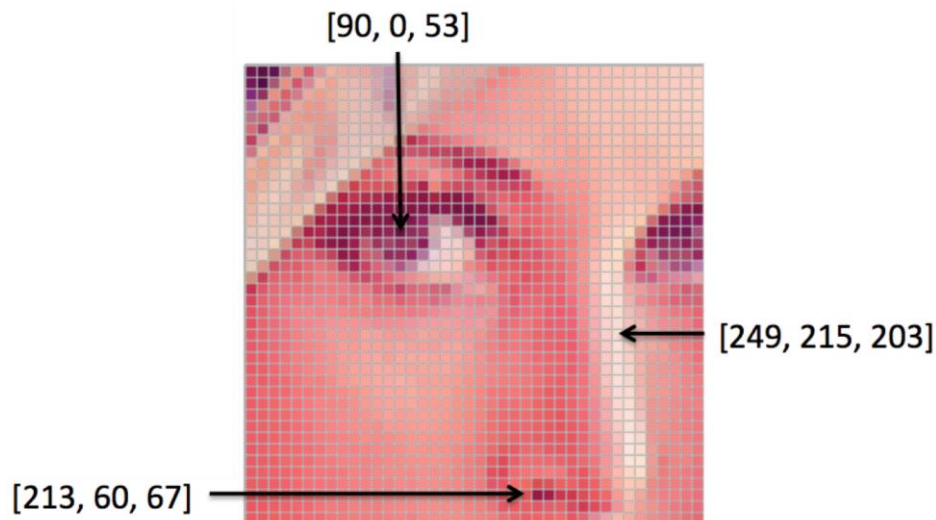
|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 131 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 237 | 239 | 239 | 228 | 227 | 87  | 71  | 201 |
| 172 | 106 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 168 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 105 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 85  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 36  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 96  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

## 3. OPENCV

### 수학적 접근 방법론

픽셀 패턴을 다루기 위해 사용되는 도구:

- 선형대수 (행렬, 벡터)
- 미분/기울기 (엣지 검출)
- 확률/통계 (노이즈, 분포)
- 최적화 (모델 학습)



$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$



### 3. OPENCV

---

$$g[n, m] = \begin{cases} 255, & f[n, m] > 100 \\ 0, & \text{otherwise.} \end{cases}$$



#### 이미지 임계처리 (Thresholding)

왼쪽 이미지의 수식  $g[n, m]$ 은  
특정 밝기 값(100)을 기준으로  
이미지를 흑백(0 또는 255)으로 나누는 이진화(Binarization) 과정

### 3. OPENCV

---

가상환경 구성 후 VSCODE 실행 > 아래 패키지 설치

```
pip install opencv-python opencv-contrib-python numpy matplotlib
```

```
import cv2

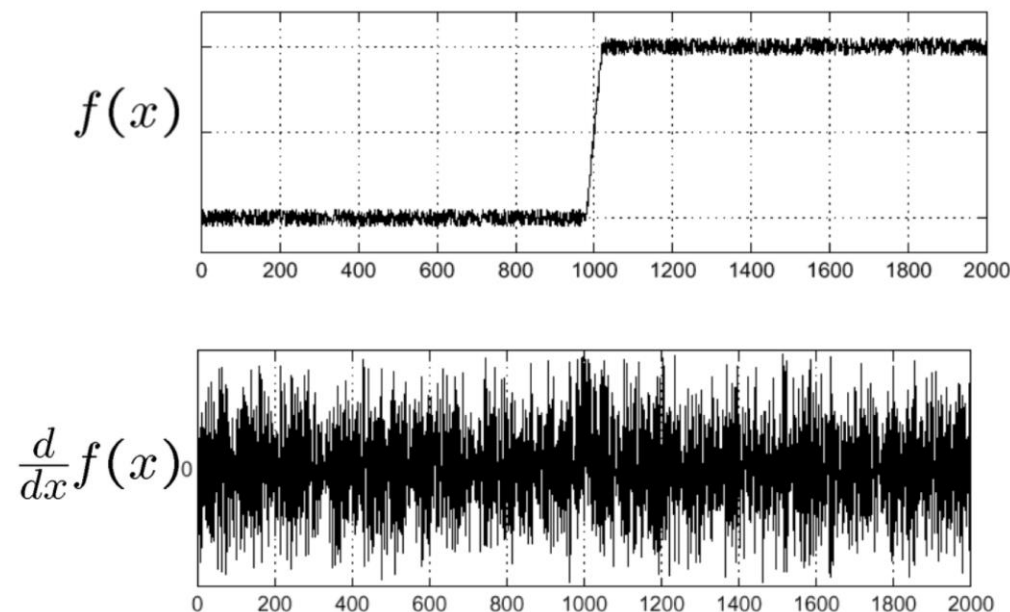
# 이미지 로드 (그레이스케일)
img = cv2.imread('lenna.png', cv2.IMREAD_GRAYSCALE)

# 수식 구현: 100보다 크면 255, 아니면 0
# cv2.threshold(src, thresh, maxval, type)
ret, thresh_img = cv2.threshold(img, 100, 255, cv2.THRESH_BINARY)

cv2.imshow('Binary Image', thresh_img)
```

### 3. OPENCV

---



#### 이미지 미분 (Derivative, Sobel)

그래프는 신호  $f(x)$ 와 그 미분값  
이미지에서 미분은 에지(Edge, 경계선)를 찾는 데 사용  
픽셀 값이 급격히 변하는 곳에서 미분값이 크게 나타나기 때문.

### 3. OPENCV

---

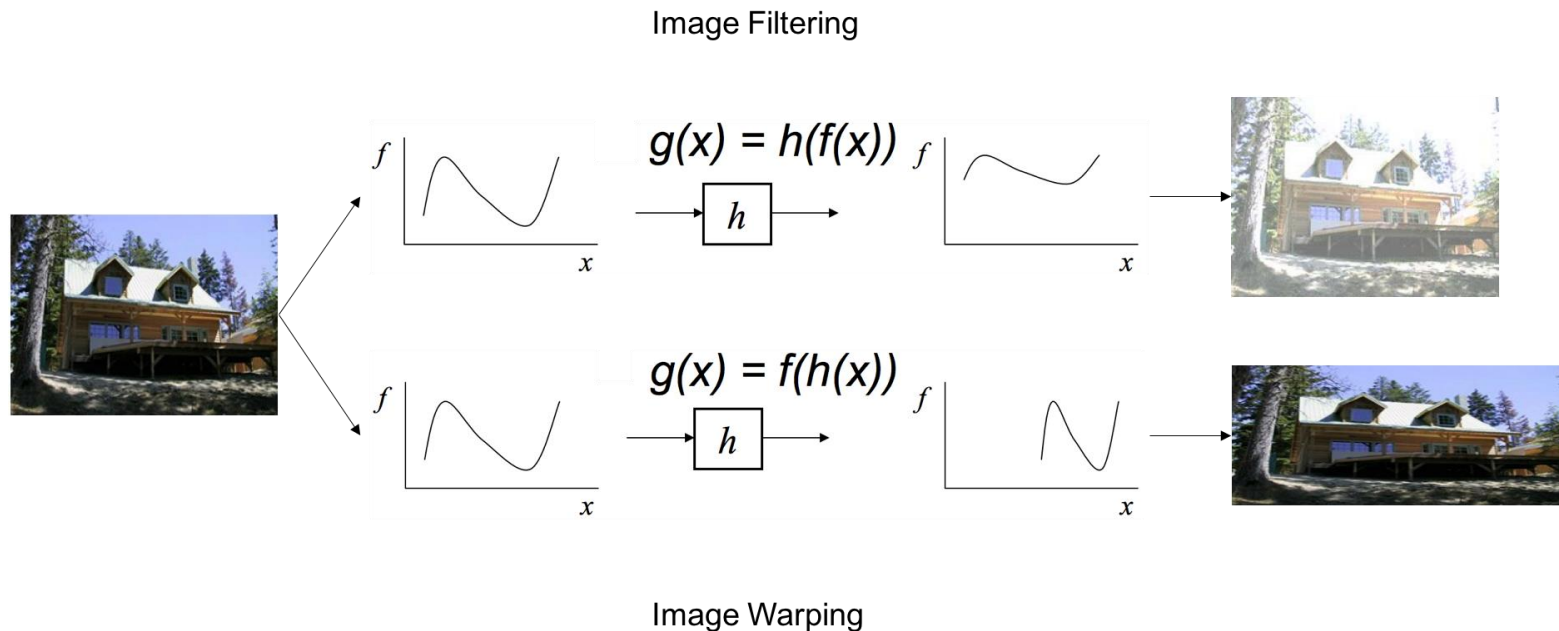
```
# OpenCV에서 이미지 미분 (Sobel 필터)  
# x축 방향 미분  
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)  
  
# 절대값을 취해 시각화  
sobel_x = cv2.convertScaleAbs(sobel_x)  
cv2.imshow('Edge Detection', sobel_x)
```

**소벨 필터** 이미지의 픽셀 밝기 값(Intensity)에 대해 미분을 수행

이미지를 하나의 거대한 행렬로 보고, 각 좌표  $(x, y)$ 에 저장된 밝기 수치가 주변 픽셀과 비교했을 때 얼마나 급격하게 변하는지를 계산  
값이 일정하다가(평탄한 곳) 갑자기 커지거나 작아지는(경계선) 지점에서 미분 결과값이 크게 나옵니다.

## 3. OPENCV

### Image Filtering (이미지 필터링)



### Image Warping (이미지 워핑)

## 3. OPENCV

---

```
import cv2
import numpy as np

img = cv2.imread('house.jpg')

# 예시: 밝기 증가 필터링 ( $g(x) = f(x) + \text{beta}$ )
# 이미지의 픽셀 값 자체를 변환함
filtering_img = cv2.convertScaleAbs(img, alpha=1.0, beta=50)

cv2.imshow('Image Filtering', filtering_img)
```

### Image Filtering (이미지 필터링)

이미지 필터링은 픽셀의 값(Value)을 변경

이미지의 위치는 그대로 두고,

해당 위치의 밝기나 색상을 함수  $h$ 에 통과시켜 변화.

이미지에서는 전체적으로 밝기가 밝아진 것을 볼 수 있습니다.

### 3. OPENCV

---

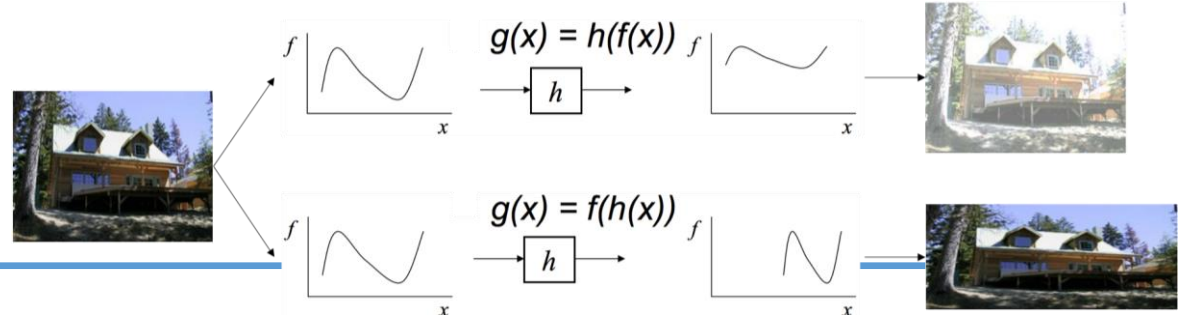
```
# 예시: 가로 크기 조절 ( $g(x) = f(h(x))$ )  
# 이미지의 좌표 공간을 변환함, shape 는 배열의 차원 정보를 갖는다  
h, w = img.shape[:2]  
  
# 가로를 0.5배로 압축하는 워핑  
warping_img = cv2.resize(img, (int(w*0.5), h),  
interpolation=cv2.INTER_LINEAR)  
  
cv2.imshow('Image Warping', warping_img)
```

#### Image Warping (이미지 워핑)

픽셀의 위치(Coordinate)를 변경

픽셀 값 자체보다는 좌표값  $x$ 에 함수  $h$ 를 적용하여 이미지를 비틀거나, 크기를 조절하거나, 회전시킵니다. 이미지에서는 집의 모양이 가로로 압축된 것을 볼 수 있습니다.

### 3. OPENCV



**$f(x)$ :** 입력 이미지의 픽셀 밝기(Intensity) 값

**$h(x)$ :** 좌표를 이동시키는 기하학적 변환 함수

| 구분          | $g(x)=h(f(x))$ (Filtering) | $g(x)=f(h(x))$ (Warping)   |
|-------------|----------------------------|----------------------------|
| 조작 대상       | 픽셀 값 (밝기, 색상)              | 픽셀 좌표 (위치, 모양)             |
| 함수 $h$ 의 역할 | 밝기를 어떻게 바꿀 것인가?            | 어디에 있는 값을 가져올 것인가?         |
| OpenCV 예시   | cv2.GaussianBlur, cv2.add  | cv2.resize, cv2.warpAffine |



### 3. OPENCV

---

‘숫자 배열(이미지)’을 쉽게 다루기 위한 도구’

→ OpenCV



### 3. OPENCV

---

| 버전  | 핵심 키워드  | 시대적 역할    |
|-----|---------|-----------|
| 1.x | 알고리즘 집합 | 비전 연구 출발점 |
| 2.x | C++화    | 라이브러리 완성  |
| 3.x | 모듈화     | 딥러닝 준비    |
| 4.x | DNN 중심  | 실전·산업 표준  |



# 3. OPENCV

---

## (2) OpenCV의 역할과 한계

이미지 처리(필터, 변환, 에지, 윤곽선),  
특징점, 카메라 보정, 기본적인 비전 파이프라인 구현

최신 딥러닝 기반 인식(Detection/Segmentation 등)은  
보통 PyTorch/TensorFlow + 모델(YOLO 등) 이 주력이 되고,

**OpenCV는 전/후처리·시각화·영상 I/O 쪽에서 많이 쓰입니다.**



# 3. OPENCV

```
import cv2
import numpy as np

# 1. 이미지 로드 및 크기 정보 획득 (image_d03275.png 내용)
img = cv2.imread('input.jpg')
h, w = img.shape[:2]

# 2. 이미지 필터링 (Filtering): 가우시안 블러
# 픽셀 값을 주변과 섞어 부드럽게 만들 ( $g(x) = h(f(x))$ )
blur_img = cv2.GaussianBlur(img, (5, 5), 0)

# 3. 이미지 워핑 (Warping): 크기 조절
# 픽셀 좌표를 이동시켜 크기를 변경 ( $g(x) = f(h(x))$ )
warped_img = cv2.resize(img, (w // 2, h // 2))

# 4. 이진화 (Thresholding): image_da2e3c.jpg 수식 구현
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
ret, thresh_img = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

# 5. 미분을 이용한 에지 검출 (Sobel Filter)
# x방향 미분값을 구해 세로선 강조
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)

# 결과 확인
cv2.imshow('Filtering (Blur)', blur_img)
cv2.imshow('Thresholding', thresh_img)
cv2.imshow('Edge (Sobel)', sobel_x)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

# 3. OPENCV

---

## 2-1) 이미지 처리(Image Processing)

- **필터링(Filtering):  $g(x) = h(f(x))$** , 픽셀의 위치는 그대로 두고 픽셀 값(밝기, 색상)을 바꿉니다. 이미지의 노이즈를 제거하거나 선명하게 만드는 작업. 주변 값들과 연산하여 노이즈를 제거하거나 선명하게 만듭니다. Blur, Gaussian, Median
- **변환(Transformation/Warping):  $g(x) = f(h(x))$**  픽셀 값은 유지하되 픽셀의 위치(좌표)를 옮깁니다. 이미지 크기 조절, 회전, 왜곡 보정 등이 이에 해당합니다. Resize, Rotate, Warp
- **색공간(Color Space):** 빛의 삼원색(RGB)이나 인간의 인지 방식(HSV)으로 색을 표현합니다, BGR  $\leftrightarrow$  RGB  $\leftrightarrow$  HSV
- **이진화(Thresholding):** 특정 기준에 따라 픽셀을 0(검정) 또는 255(흰색)로 나눕니다. Threshold, Adaptive Threshold

>> 픽셀 수준의 조작

# 3. OPENCV

```
import cv2
import numpy as np

# 1. 이미지 로드 및 크기 정보 획득 (image_d03275.png 내용)
img = cv2.imread('input.jpg')
h, w = img.shape[:2]

# 2. 이미지 필터링 (Filtering): 가우시안 블러
# 픽셀 값을 주변과 섞어 부드럽게 만듦 ( $g(x) = h(f(x))$ )
blur_img = cv2.GaussianBlur(img, (5, 5), 0)

# 3. 이미지 워핑 (Warping): 크기 조절
# 픽셀 좌표를 이동시켜 크기를 변경 ( $g(x) = f(h(x))$ )
warped_img = cv2.resize(img, (w // 2, h // 2))

# 4. 이진화 (Thresholding): image_da2e3c.jpg 수식 구현
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
ret, thresh_img = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

# 5. 미분을 이용한 에지 검출 (Sobel Filter)
# x방향 미분값을 구해 세로선 강조
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)

# 결과 확인
cv2.imshow('Filtering (Blur)', blur_img)
cv2.imshow('Thresholding', thresh_img)
cv2.imshow('Edge (Sobel)', sobel_x)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

## 3. OPENCV

### 2-2) 에지 & 윤곽선 분석

- Edge Detection (Canny 등)
- Contour 추출: 연결된 에지 점들을 찾아 하나의 폐곡선(외곽선)으로 만듭니다.
- 형태 분석: 추출된 윤곽선을 이용해 객체의 면적, 둘레, Bounding Box(상자 크기) 등을 계산하여 객체의 구조를 파악합니다.



(a) Original



(b) Canny,  
 $\sigma = 1$



(c) Canny,  
 $\sigma = 3$

# 3. OPENCV

---

## 2-3) 특징점 & 기하 기반 비전

- 특징점: ORB, SIFT, SURF매칭, 파노라마
  - 카메라 보정(Camera Calibration)
  - 좌표 변환(Affine, Perspective)
- 기하학적 해석과 정렬 문제에 강함

### **SIFT (Scale-Invariant Feature Transform)**

이미지의 크기 변화, 회전, 밝기 변화에 매우 강력합니다.

단점: 연산량이 매우 많아 실시간 처리에 불리, 특허권 문제.

### **SURF (Speeded-Up Robust Features)**

SIFT의 속도 문제를 개선.

### **ORB (Oriented FAST and Rotated BRIEF)**

무료(Open Source) 알고리즘.

연산 속도가 매우 빨라 모바일/실시간 영상에 가장 많이 쓰입니다.



# 3. OPENCV

---

```
import cv2

# 1. 이미지 로드 및 그레이스케일 변환
img = cv2.imread('scene.jpg', cv2.IMREAD_GRAYSCALE)

# 2. ORB 탐지기 생성
orb = cv2.ORB_create()

# 3. 특징점(Keypoints) 검출 및 기술자(Descriptors) 계산
# descriptors는 특징점 주변의 정보를 숫자로 담고 있어 '매칭'에 쓰입니다.
keypoints, descriptors = orb.detectAndCompute(img, None)

# 4. 특징점 시각화
# 이미지 위에 찾은 점들을 원으로 그려줍니다.
result_img = cv2.drawKeypoints(img, keypoints, None, color=(0, 255, 0), flags=0)

cv2.imshow('ORB Features', result_img)
cv2.waitKey(0)
```

# 3. OPENCV

---

## 2-4) 비전 파이프라인 구현

단순한 이미지 처리 라이브러리를 넘어,  
카메라로부터 영상을 받고(Input), 이를 처리(Process)한 뒤,  
화면에 보여주거나 저장(Output)하는 전체 파이프라인을 구축하는 데 최적화

- **영상 입력(VideoCapture)**

`cap = cv2.VideoCapture(0):` 0번은 노트북 내장 캠/ 첫 번째 USB 카메라.

`ret, frame = cap.read():` ret은 읽기 성공 여부(True/False),  
frame은 실제 이미지 행렬(Numpy Array).

### 3. OPENCV

---

```
import cv2
```

```
cap = cv2.VideoCapture(0) # 0: 웹캠, "video.mp4": 파일
```

```
if not cap.isOpened():
```

```
    raise RuntimeError("비디오를 열 수 없습니다.")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

### 3. OPENCV

---

- **처리결과 시각화(Drawing) : rectangle, circle, line, putText**

`cv2.rectangle()`: 사각형 박스 그리기

`cv2.putText()`: 텍스트 정보 쓰기

- **화면 출력 (imshow)**

- **파일 저장/로드**

- **자원 해제**

`cv2.waitKey(1)`: 키보드 입력을 기다리는 함수로,  
영상 출력 시 프레임 속도를 조절하는 역할도 합니다.

👉 End-to-End 비전 실험의 뼈대 역할

## 3. OPENCV

---

- 처리결과 시각화(Drawing)

```
edges = cv2.Canny(gray, 100, 200)

cv2.rectangle(frame, (50,50), (200,200), (0,255,0), 2)
cv2.putText(frame, "Edge Detection",
            (30,30),
            cv2.FONT_HERSHEY_SIMPLEX,
            1, (0,0,255), 2)
```

- 화면 출력 (imshow)

```
cv2.imshow("Result", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

- 파일 저장

```
cv2.imwrite("output.jpg", frame)
```

# 3. OPENCV

---

- 영상 저장

```
fourcc = cv2.VideoWriter_fourcc(*"XVID")  
out = cv2.VideoWriter("output.avi", fourcc, 30, (640,480))
```

- 자원 해제

```
cap.release()  
out.release()  
cv2.destroyAllWindows()
```

# 3. OPENCV

---

```
import cv2

# 1. 영상 입력 설정 (0번 카메라 또는 'video.mp4' 경로)
cap = cv2.VideoCapture(0)

# 영상 저장을 위한 설정 (선택 사항)
fourcc = cv2.VideoWriter_fourcc(*'XVID') # 코덱 설정
out = cv2.VideoWriter('output.avi', fourcc, 20.0, (640, 480))

print("영상을 시작합니다. 종료하려면 'q'를 누르세요.")

while cap.isOpened():
    # 프레임 단위로 읽기
    ret, frame = cap.read()
    if not ret:
        break

    # -----
    # [Process] 이 부분에 딥러닝 모델 예측 코드가 들어갑니다.
    # 예: 예측된 숫자가 '7'이라고 가정
    label = "Predict: 7"
    # -----
```

```
# 2. 시각화 (Drawing)
# 사각형 그리기 (이미지, 좌상단, 우하단, 색상BGR, 두께)
cv2.rectangle(frame, (200, 150), (440, 330), (0, 255, 0), 2)

# 텍스트 쓰기 (이미지, 내용, 좌표, 폰트, 크기, 색상, 두께)
cv2.putText(frame, label, (200, 140),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

# 3. 파일 저장 및 화면 출력
out.write(frame) # 파일에 저장
cv2.imshow('Vision Pipeline Test', frame) # 화면에 표시

# 종료 조건 ('q' 키를 누르면 루프 탈출)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# 자원 해제
cap.release()
out.release()
cv2.destroyAllWindows()
```

## 3. OPENCV

---

### (3) 이미지의 수학적 표현

- 그레이스케일:  $H \times W$  2D 배열 (각 원소가 밝기)
- 컬러(BGR/RGB):  $H \times W \times 3$ , 3D 배열 (채널 3개)

**픽셀 값 범위(일반적): uint8  $\rightarrow$  0~255**

(딥러닝으로 가면 **float32 + 0~1 정규화**를 자주 합니다.)



# 3. OPENCV

| 이미지 유형       | 차원 (Shape)                              | 배열 크기 예시                           | 각 원소 의미                   | 데이터 타입 (일반적) | 값 범위      | 용도                     |
|--------------|-----------------------------------------|------------------------------------|---------------------------|--------------|-----------|------------------------|
| 그레이스케일       | 2D (H × W)                              | (480, 640)                         | 밝기 (intensity)            | uint8        | 0 ~ 255   | 간단한 처리, 메모리 절약         |
| 컬러 (BGR/RGB) | 3D (H × W × 3)                          | (480, 640, 3)                      | Blue, Green, Red (또는 RGB) | uint8        | 0 ~ 255   | 컬러 이미지                 |
| 딥러닝 입력       | 3D or 4D (H × W × C) or (N × H × W × C) | (224, 224, 3) or (32, 224, 224, 3) | 정규화된 픽셀 값                 | float32      | 0.0 ~ 1.0 | PyTorch, TensorFlow 학습 |

### 3. OPENCV

---

```
import cv2
import numpy as np

# 1. 그레이스케일로 읽기
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
print(gray.shape)    # 예: (720, 1280)
print(gray.dtype)    # uint8
print(gray[100, 200]) # → 157 (한 픽셀의 밝기 값)

# 2. 컬러(BGR)로 읽기 ← OpenCV 기본
color = cv2.imread("photo.jpg")    # flag 생략 = BGR
print(color.shape)    # 예: (720, 1280, 3)
print(color.dtype)    # uint8
print(color[100, 200]) # → [ 89 124 201] ← B, G, R 순서!
```

### 3. OPENCV

# 3. 채널 따로 떼기

```
B, G, R = cv2.split(color)
```

# 또는

```
B = color[:, :, 0]
```

```
G = color[:, :, 1]
```

```
R = color[:, :, 2]
```

# 결과 시각화 (각 채널을 그레이스케일 형태로 확인)

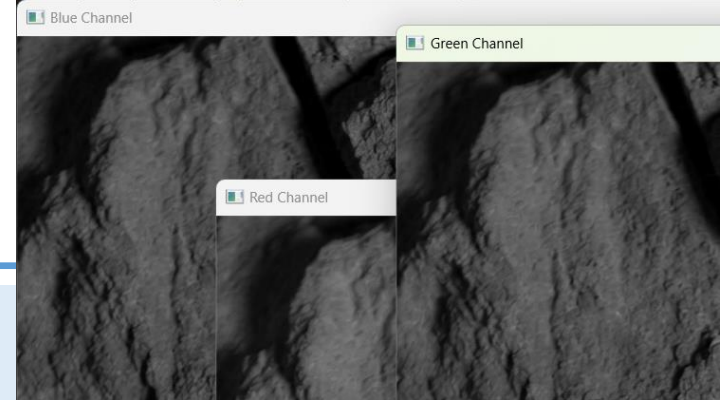
```
cv2.imshow('Blue Channel', B) # 파란색이 강할수록 해당  
부분이 밝게 보임
```

```
cv2.imshow('Green Channel', G)
```

```
cv2.imshow('Red Channel', R)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



분리된 채널 (B 또는 G 또는 R):

cv2.split()이나 슬라이싱을 거치면 (세로, 가로)만 남은  
2차원 배열이 됩니다.

채널 정보가 1개뿐이므로 '밝기(Intensity) 값'으로만 해석.

### 3. OPENCV

---

| 단계            | 데이터 타입  | 값 범위                              | 왜 이렇게?                                                                     |
|---------------|---------|-----------------------------------|----------------------------------------------------------------------------|
| OpenCV 기본     | uint8   | 0 ~ 255                           | 8비트 정수 → 파일 저장 효율적, 인간 눈에 맞춤                                               |
| 딥러닝 입력 전      | float32 | 0.0 ~ 1.0                         | 신경망은 작은 숫자 + 미분이 잘 되는 범위에서 학습이 안정적                                         |
| PyTorch/TF 표준 | float32 | 0.0 ~ 1.0                         | 대부분의 pretrained 모델이 이 범위를 가정 (ImageNet 등)                                  |
| 추가 정규화        | float32 | mean $\approx$ 0, std $\approx$ 1 | $(x - \text{mean}) / \text{std}$ → 학습 더 빨라지고 안정적 (transforms.Normalize 사용) |

### 3. OPENCV

---

# 1. 가장 간단한 0~1 정규화

```
img_float = color.astype(np.float32) / 255.0
print(img_float.shape)  # (720, 1280, 3)
print(img_float.dtype)  # float32
print(img_float[100, 200]) # 예: [0.349 0.486 0.788] ← 0~1 사이
```

# 2. PyTorch 스타일 (transforms 사용 시)

```
# mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225] ← ImageNet 값
normalized = (img_float - np.array([0.485, 0.456, 0.406])) / np.array([0.229, 0.224, 0.225])
```

# 3. OPENCV

---

## 자주 하는 실수

1 : **OpenCV는 BGR**, matplotlib/numpy/PIL은 RGB

→ 색이 뒤집혀 보임해결

→ `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

2 : uint8 그대로 딥러닝에 넣음

→ 학습 안 됨

**항상 / 255.0 + `.astype(np.float32)` 해주기**

3 : **shape 확인** 안 함

→ (H,W,C)인지 (C,H,W)인지 헷갈림 (PyTorch는 채널이 앞)

```
img = cv2.imread(path)
if img is None:
    raise FileNotFoundError("이미지 로드 실패")
print(f"Shape: {img.shape}, Dtype: {img.dtype}")
```

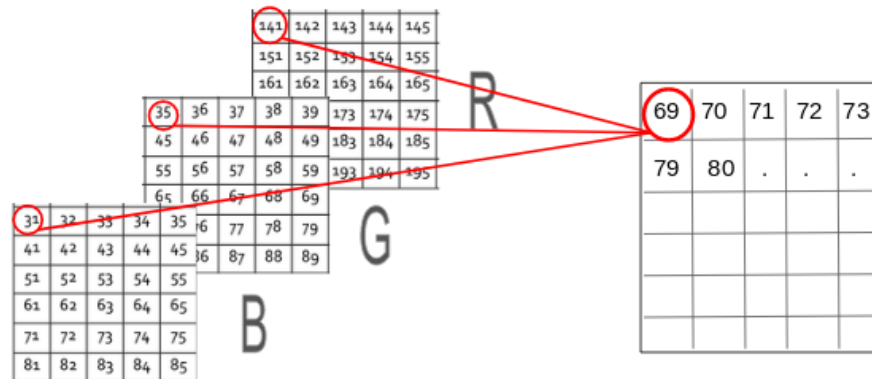
### 3. OPENCV

#### Color에서 Gray변환의 수학적 의미 : BGR2GRAY, GBR2GRAY

인간의 눈은 초록(G)에 가장 민감

- $\text{Gray} \approx 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$
- 색의 평균이 아니라 **'지각된 밝기'의 근사값**
- 그레이 변환은 색(Color)을 버리고, 인간이 느끼는 '밝기(Luminance)'만 남기는 과정

- 1) 각 픽셀에 대해 B, G, R 분리
- 2) 내부적으로 BGR → RGB 순서 보정
- 3) 가중합 적용결과를 1채널 이미지로 저장



## 3. OPENCV

### Color conversions

See `cv::cvtColor` and `cv::ColorConversionCodes`

Todo

document other conversion modes

### RGB <-> GRAY

Transformations within RGB space like adding/removing the alpha channel, reversing the channel order, conversion to/from 16-bit RGB color (R5:G6:B5 or R5:G5:B5), as well as conversion to/from grayscale using:

$$\text{RGB[A] to Gray: } Y \leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

and

$$\text{Gray to RGB[A]: } R \leftarrow Y, G \leftarrow Y, B \leftarrow Y, A \leftarrow \max(\text{ChannelRange})$$

The conversion from a RGB image to gray is done with:



## 3. OPENCV

### 좌표와 배열 인덱싱

이미지 접근은 항상: **img[y, x]**  
y: 행(row) → 세로  
x: 열(column) → 가로

수학 좌표(x, y)와 순서가 반대, NumPy와 동일한 메모리 모델

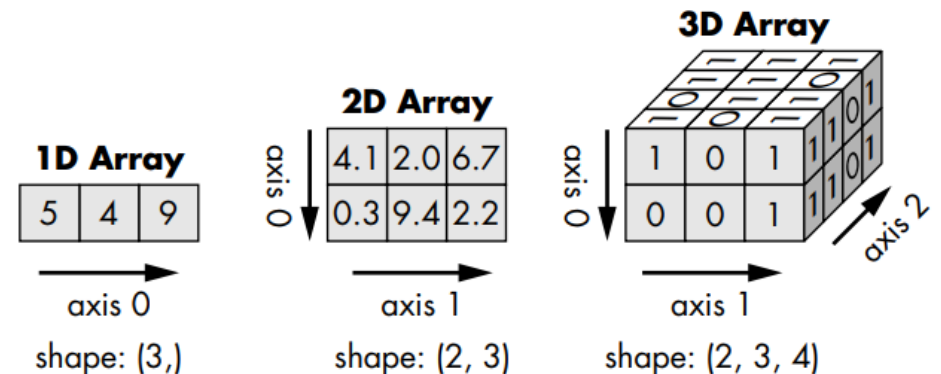
행 우선(row-major) 저장

“몇 번째 줄(row)인지 먼저, 그 줄의 몇 번째 칸인지”

# (x=10, y=20)에 해당하는 픽셀

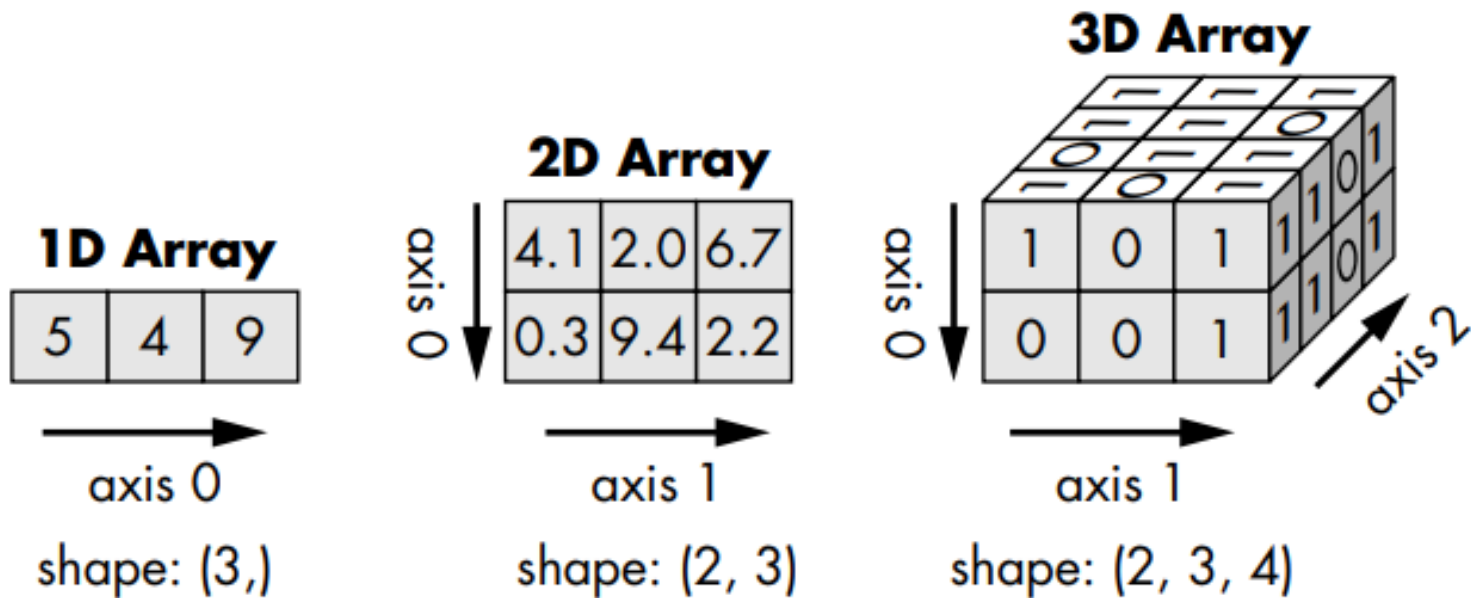
pixel = img[20, 10]

ROI 슬라이싱 : img[y1:y2, x1:x2]



### 3. OPENCV

---



### 3. OPENCV

---

## OPENCV 실습 예시코드

# 가상환경 활성화 상태에서 쥬피터 확장팩을 설치 후 .ipynb로 파일 실행  
# 아래 커널 설치 후 실행

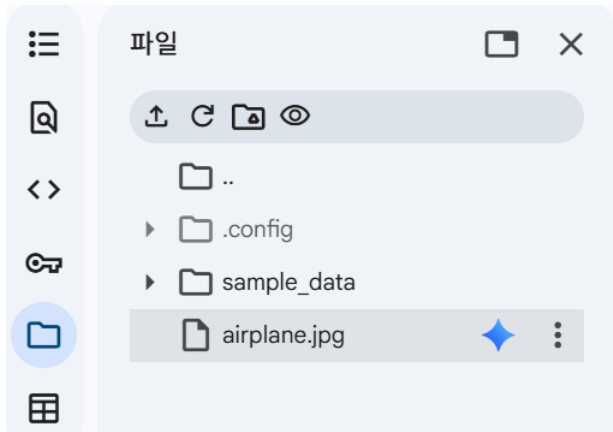
```
pip install ipykernel jupyter
```

## 3. OPENCV

### Pillow(PIL : Python Imaging Library)

Python에서 이미지를 다루기 위한 라이브러리

- 이미지 읽기 / 쓰기
- 이미지 형식 변환
- 간단한 이미지 조작



```
from PIL import Image
import numpy as np

img = Image.open("image.jpg")
img_np = np.array(img)

print(img_np.shape, img_np.dtype)
```

# 3. OPENCV

---

## Matplotlib출력

(viewer) 용도로 최적

```
import matplotlib.pyplot as plt

plt.imshow(img_np)
plt.axis("off")
plt.show()
```



## 3. OPENCV

---

### OpenCV

고성능 이미지/영상 처리 라이브러리  
기본 색상 순서: BGR  
로컬 Python 스크립트에서 강력

```
import cv2  
  
img_bgr = cv2.imread("image.jpg")  
print(img_bgr.shape, img_bgr.dtype)
```

```
type: <class 'numpy.ndarray'>  
shape (H, W, C): (427, 640, 3)  
dtype: uint8 min/max: 0 255
```

### 3. OPENCV

---

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
# 1) 이미지 읽기  
# 파일 경로를 본인 환경에 맞게 수정하세요.  
img_bgr = cv2.imread("/content/airplane.jpg")  
  
if img_bgr is None:  
    raise FileNotFoundError("이미지를 읽지 못했습니다. 경로/파일명을 확인하세요.")  
  
# 2) 이미지 속성 확인  
print("type:", type(img_bgr))  
print("shape (H, W, C):", img_bgr.shape)  
print("dtype:", img_bgr.dtype)  
print("min/max:", img_bgr.min(), img_bgr.max())
```

### 3. OPENCV

---

```
# 3) BGR -> RGB 변환 (색 정상화)
```

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(7, 4))
```

```
plt.imshow(img_rgb)
```

```
plt.title("Loaded Image (RGB for display)")
```

```
plt.axis("off")
```

```
plt.show()
```





### 3. OPENCV

# 4) 특정 픽셀 값 확인 (y, x) 순서 주의!

y, x = 10, 10

b, g, r = img\_bgr[y, x]

print(f"Pixel at (y={y}, x={x}) in BGR:", (int(b), int(g), int(r)))

# 5) 픽셀 값 수정 (해당 픽셀을 '빨강'으로)

# OpenCV는 BGR이므로 빨강은 (0, 0, 255)

img\_edit = img\_bgr.copy()

img\_edit[y, x] = (0, 0, 255)

img\_edit\_rgb = cv2.cvtColor(img\_edit, cv2.COLOR\_BGR2RGB)

plt.figure(figsize=(7, 4))

plt.imshow(img\_edit\_rgb)

plt.title("Edited Image (ROI + Pixel)")

plt.axis("off")

plt.show()



Edited Image (ROI + Pixel)



### 3. OPENCV

---

```
# 6) ROI(사각 영역) 수정: 좌상단 0:80, 0:200 영역을 초록색으로 칠하기
```

```
# 초록(BGR) = (0, 255, 0)
```

```
img_edit[0:80, 0:200] = (0, 255, 0)
```

```
# 7) 수정 결과 확인 (RGB로 변환해서 보기)
```

```
img_edit_rgb = cv2.cvtColor(img_edit, cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(7, 4))
```

```
plt.imshow(img_edit_rgb)
```

```
plt.title("Edited Image (ROI + Pixel)")
```

```
plt.axis("off")
```

```
plt.show()
```

```
# 8) 저장
```

```
out_path = lesson1_edited.png"
```

```
cv2.imwrite(out_path, img_edit)
```

```
print("Saved to:", out_path)
```

Edited Image (ROI + Pixel)



# 3. OPENCV

---

## 1) cv2.imread()

**img\_bgr = cv2.imread("image.png")**

디스크에 있는 이미지 파일을 메모리로 읽어 NumPy 배열(numpy.ndarray) 로 반환

기본 컬러 순서:

BGR파일을 못 찾거나 읽기 실패 시 None 반환

OpenCV는 예외를 던지지 않고 None을 돌려주기 때문에 직접 오류 처리 필수

```
if img_bgr is None:  
    raise FileNotFoundError(...)
```

# 3. OPENCV

---

## 2) `img.shape`, `img.dtype`, `img.min()` / `max()`

NumPy 배열의 기본 속성 확인

OpenCV 전용이 아니라 NumPy 표준의미

- **shape**: (H, W, C) → 해상도와 채널 수
- **dtype**: 보통 `uint8`
- **min/max**: 픽셀 값 범위 확인 (0~255)

딥러닝 전처리 시 `uint8` → `float32`,

0~255 → 0~1 변환 판단 기준

```
img_bgr.shape  
img_bgr.dtype  
img_bgr.min(), img_bgr.max()
```

## 3. OPENCV

---

### 3) cv2.cvtColor()

색공간(Color Space) 변환, 여기서는 BGR → RGB

- OpenCV: BGR
- Matplotlib / PIL: RGB → 변환 안 하면 색이 뒤집혀 보임

각 픽셀의 채널 순서를 재배치하는 연산

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

## 3. OPENCV

---

### 4) cv2.COLOR\_BGR2RGB

cvtColor에 전달하는 변환 플래그 상수

“입력은 BGR, 출력은 RGB”

다양한 변환 조합(BGR↔GRAY, BGR↔HSV, BGR↔LAB 등)

```
cv2.COLOR_BGR2RGB
```

## 3. OPENCV

---

### 5) `img[y, x]` (픽셀 접근)

특정 위치의 픽셀 값 직접 접근

`img[y, x]`

**y: 행(row) → 세로**

**x: 열(column) → 가로**

수학 좌표 (x, y)와 순서 반대

```
b, g, r = img_bgr[y, x]
```

## 3. OPENCV

---

### 6) `img.copy()`

깊은 복사(deep copy)

원본 이미지 보호

✗ 같은 메모리 참조, 이렇게 하면 수정 시 원본도 함께 바뀜  
(`img_edit = img_bgr` )

```
img_edit = img_bgr.copy()
```



## 3. OPENCV

---

### 7) ROI 슬라이싱 (img[y1:y2, x1:x2])

**관심 영역(ROI, Region of Interest)** 을 한 번에 선택

- 해당 영역의 모든 픽셀 값을 변경
- NumPy 슬라이싱 규칙 그대로 사용
- OpenCV의 ROI는 새로운 개념이 아니라 배열 연산

```
img_edit[0:80, 0:200] = (0, 255, 0)
```

## 3. OPENCV

---

### 8) cv2.imwrite()

NumPy 배열을 이미지 파일로 저장

- 확장자(.png, .jpg)로 포맷 자동 결정
- 성공 시 True, 실패 시 False 반환

```
cv2.imwrite(out_path, img_edit)
```

### **3. OPENCV**

---

## **주요 명령어 파라미터**

## 3. OPENCV

---

 **OpenCV** 4.14.0-dev  
Open Source Computer Vision

[Main Page](#) [Related Pages](#) [Namespaces ▾](#) [Classes ▾](#) [Files ▾](#) [Examples](#) [Java documentation](#)

**◆ imread()** [2/2]

```
void cv::imread ( const String & filename,  
                 OutputArray dst,  
                 int flags = IMREAD_COLOR_BGR )
```

**Python:**  

```
cv.imread( filename[, flags] ) -> retval  
cv.imread( filename[, dst[, flags]] ) -> dst
```

```
#include <opencv2/imgcodecs.hpp>
```

Loads an image from a file.

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts and the return value.

**Parameters**  

|                 |                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------|
| <b>filename</b> | Name of file to be loaded.                                                                       |
| <b>dst</b>      | object in which the image will be loaded.                                                        |
| <b>flags</b>    | Flag that can take values of <b>cv::ImreadModes</b> , default with <b>cv::IMREAD_COLOR_BGR</b> . |

# 3. OPENCV

---

## ① cv2.imread() — 이미지 읽기

### 1. 기본 형태

`img = cv2.imread(filename, flags=cv2.IMREAD_COLOR)`

반환값: numpy.ndarray 또는 실패 시 None

기본 컬러 순서: BGR

**중요: 실패해도 예외를 던지지 않음 → None 체크 필수**

**if img is None:**

**raise FileNotFoundError("이미지를 읽지 못했습니다.")**

# 3. OPENCV

---

## 2. 주요 파라미터 (flags)

### (1) **cv2.IMREAD\_COLOR** (기본값)

컬러로 읽기

알파 채널(투명도) 무시

```
img = cv2.imread("a.png", cv2.IMREAD_COLOR)
```

### (2) **cv2.IMREAD\_GRAYSCALE**

흑백(밝기)로 읽기

→ H x W

```
gray = cv2.imread("a.png", cv2.IMREAD_GRAYSCALE)
```

### (3) **cv2.IMREAD\_UNCHANGED**

원본 그대로 읽기

알파 채널 유지 (RGBA → BGRA)

```
img = cv2.imread("a.png", cv2.IMREAD_UNCHANGED)
```

### (4) 숫자 플래그 (동치)

1 = COLOR, 0 = GRAYSCALE, -1 = UNCHANGED (가독성 때문에 상수 사용 권장)

# 3. OPENCV

---

## 3. 자주 쓰는 변형 패턴

### (1) 컬러로 읽고 RGB로 변환

```
img_bgr = cv2.imread("a.jpg")  
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

### (2) 알파 채널 있는 PNG 읽기

```
img_bgra = cv2.imread("a.png", cv2.IMREAD_UNCHANGED)  
print(img_bgra.shape) # (H, W, 4)
```

### (3) 경로에 한글/공백 있을 때(윈도우 이슈 회피)

```
import numpy as np  
img = cv2.imdecode(np.fromfile("한글경로.png", np.uint8), cv2.IMREAD_COLOR)
```

## 3. OPENCV

---

 **OpenCV** 4.14.0-dev ▾  
Open Source Computer Vision

[Main Page](#) [Related Pages](#) [Namespaces ▾](#) [Classes ▾](#) [Files ▾](#) [Examples](#) [Java documentation](#)

### ◆ imwrite()

```
bool cv::imwrite ( const String &      filename,
                  InputArray          img,
                  const std::vector< int > & params = std::vector< int >() )
```

**Python:**

```
cv.imwrite( filename, img[, params] ) -> retval
```

```
#include <opencv2/imgcodecs.hpp>
```

Saves an image to a specified file.

The function `imwrite` saves the image to the specified file. The image format is chosen based on the filename extension (see `cv::imread` for the list of extensions). In general, only 8-bit unsigned (CV\_8U) single-channel or 3-channel (with 'BGR' channel order) images can be saved using this function, with these exceptions:



# 3. OPENCV

---

## ② cv2.imwrite() — 이미지 저장

### 1. 기본 형태

success = **cv2.imwrite(filename, img)**

반환값: True/False, 포맷 결정: 파일 확장자(.png, .jpg 등)

```
if not cv2.imwrite("out.png", img):  
    raise IOError("저장 실패")
```

# 3. OPENCV

---

## 2. 주요 파라미터 (압축/품질 옵션)

### (1) JPEG 품질

```
cv2.imwrite("out.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])  
# 0~100, 값 ↑ → 화질 ↑, 용량 ↑
```

### (2) PNG 압축 레벨

```
cv2.imwrite("out.png", img, [cv2.IMWRITE_PNG_COMPRESSION, 3])  
# 0~9, 0: 빠름/용량 큼, 9: 느림/용량 작음
```

### (3) 알파 채널 포함 저장

```
# BGRA 배열이면 알파 채널 유지 저장  
cv2.imwrite("out.png", img_bgra)
```

# 3. OPENCV

---

## 3. 자주 쓰는 변형 패턴

(1) RGB 이미지를 OpenCV로 저장할 때

# matplotlib/PIL에서 온 RGB라면 BGR로 바꿔 저장

```
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)  
cv2.imwrite("out.jpg", img_bgr)
```

(2) ROI만 잘라 저장

```
roi = img[100:300, 200:500]  
cv2.imwrite("roi.png", roi)
```

(3) 디렉터리 자동 생성

```
import os  
os.makedirs("outputs", exist_ok=True)  
cv2.imwrite("outputs/out.png", img)
```

### 3. OPENCV

---

#### 4. dtype/범위 주의 (중요)

**OpenCV 저장은 주로 uint8 (0~255) 기준**

딥러닝 출력(float32, 0~1)은 변환 후 저장

```
img_u8 = (img_float * 255).clip(0,255).astype("uint8")
```

```
cv2.imwrite("out.png", img_u8)
```

## 3. OPENCV

---

### 미니 과제)

- 1) 이미지의 가운데 점 ( $H//2$ ,  $W//2$ ) 픽셀 값을 출력해 보세요.
- 2) ROI를 이미지 오른쪽 아래에 50x50 크기로 잡고, 그 영역을 파란색( $BGR=(255,0,0)$ )으로 칠해 보세요.
- 3) 수정본을 파일로 저장해서 공유하세요.

# 3. OPENCV

---

```
import cv2

# 1. 이미지 로드
img = cv2.imread("input.jpg") # 입력 이미지 경로
if img is None:
    raise ValueError("이미지를 불러올 수 없습니다.")

H, W, C = img.shape

# 2. 이미지 중앙 픽셀 값 출력
center_y, center_x = H // 2, W // 2
center_pixel = img[center_y, center_x]
print(f"Center pixel value (BGR): {center_pixel}")

# 3. 오른쪽 아래 50x50 ROI 설정
roi_h, roi_w = 50, 50
y_start = H - roi_h
x_start = W - roi_w

# ROI를 파란색으로 채우기 (BGR = (255, 0, 0))
img[y_start:H, x_start:W] = (255, 0, 0)

# 4. 결과 이미지 저장
output_path = "output_roi_blue.jpg"
cv2.imwrite(output_path, img)

print(f"수정된 이미지 저장 완료: {output_path}")
```

# 3. OPENCV

---

```
import cv2

# 1. 이미지 로드
img = cv2.imread('input.jpg')

if img is None:
    print("이미지 파일을 찾을 수 없습니다.")
else:
    # 2. 이미지 크기 정보 획득
    # h: 세로(height), w: 가로(width)
    h, w = img.shape[:2]

    # 3. 가운데 점 (H/2, W/2) 픽셀 값 출력
    center_pixel = img[h//2, w//2]
    print(f"중심점 (y:{h//2}, x:{w//2})의 BGR 값: {center_pixel}")

    # 4. 오른쪽 아래 ROI 설정 (50x50 크기)
    roi = img[h-50:h, w-50:w]

    # 5. 해당 영역을 파란색(BGR=(255, 0, 0))으로 칠하기
    img[h-50:h, w-50:w] = [255, 0, 0]

    # 6. 수정본 파일 저장
    cv2.imwrite('modified_image.jpg', img)

    # 7. 결과 확인 (시각화)
    cv2.imshow('Modified Image', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

# 3. OPENCV

---

## 2차시 — 색공간 & 밝기 처리 (보는 방식 바꾸기)



### 차시 목표

- 색을 분해해서 사고하기
- "색 vs 밝기"의 역할 이해



### 주요 주제

- Color Space
- Grayscale의 의미
- HSV의 철학



### 필수 이론 & 용어

- BGR / RGB / Grayscale / HSV
- Hue / Saturation / Value
- 밝기 기반 처리의 이유



### 필수 실습

- 색공간 변환
- HSV로 특정 색 영역 마스킹



# 3. OPENCV

---

## 1) Color Space

Color Space(색공간): 색을 수학적으로 표현하는 좌표계

대표 색 공간:

- **Grayscale: 밝기 중심**

밝기 정보만 남겨 연산량을 줄이며, 에지 검출 등의 기초 단계로 활용

- **RGB / BGR: 장치 친화적(디스플레이)**

디스플레이 장치 친화적이며, OpenCV는 기본적으로 BGR 순서를 사용

- **HSV: 인간 인식 친화적(색/밝기 분리)**

인간의 인식과 유사하며, 특정 색상 영역을 검출할 때 매우 유리

## 3. OPENCV

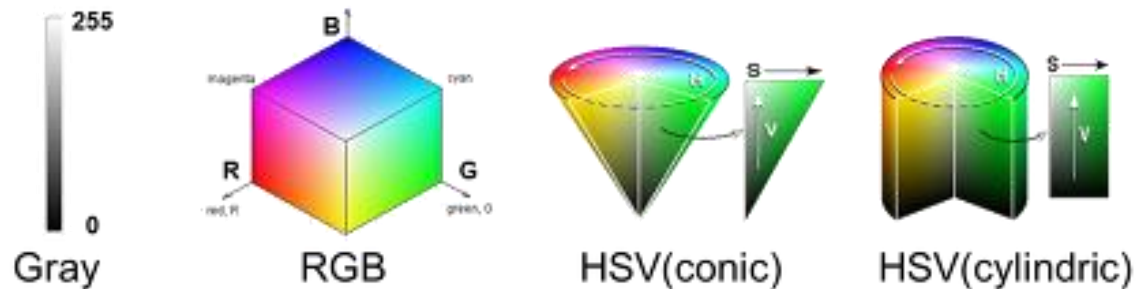
---

### 2) Grayscale의 의미

색 정보를 제거하고 **밝기(Intensity)** 만 남김

**수학적으로 단순 → 연산 안정적 → 노이즈에 강함**

Edge Detection, Thresholding, Contour같은 기법은 **그레이스케일이 기본 입력**

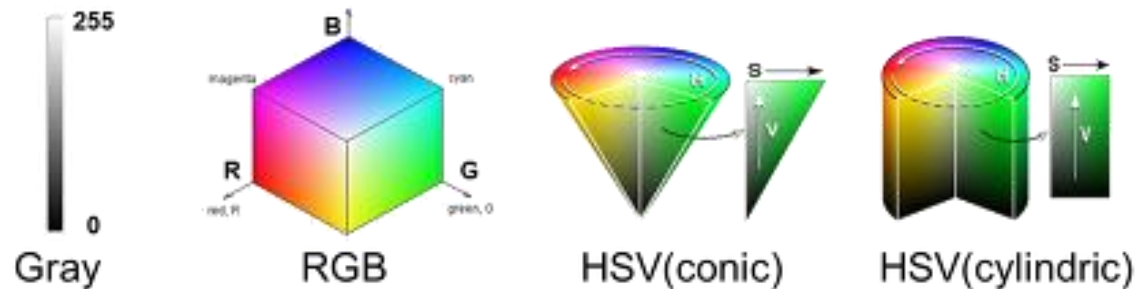


# 3. OPENCV

## 3) RGB 색공간

빛의 삼원색인 Red(빨강), Green(초록), Blue(파랑)를 조합하여 색을 표현

- **가산 혼합**: 색을 섞을수록 밝아지며, 모두 최대치로 섞으면 흰색이 됩니다.
- **데이터 구조**: 각 픽셀은 **3개의 채널(R, G, B)** 정보를 가지며, 각 채널은 0~255(uint8).
- **빛의 밝기 변화에 매우 취약**. 같은 빨간색 사과라도 그림자가 지면 R, G, B 값이 모두 변하기 때문에 컴퓨터가 같은 색으로 인식하기 어렵다.



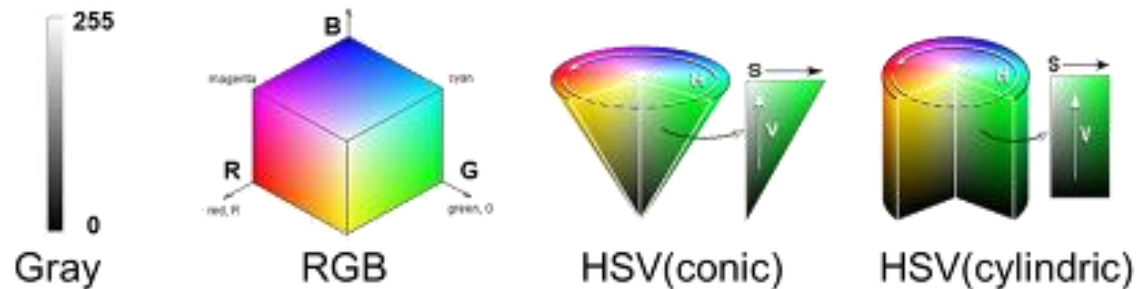
# 3. OPENCV

## 4) HSV의 철학

“어떤 색 계열이고, 얼마나 진하고, 얼마나 밝은가?”

- Hue (H): 색상(빨강/파랑/초록 계열)
- Saturation (S): 색의 진함
- Value (V): 밝기

색 검출은 HSV가 압도적으로 유리



## 3. OPENCV

---

### 주요 명령어 파라미터

## 3. OPENCV

---

### ① cv2.cvtColor() — 색공간 변환

`dst = cv2.cvtColor(src, code)`  
**이미지의 색공간(Color Space) 을 변환**

주요 파라미터

- `src` : 입력 이미지 (보통 BGR)
- `code` : 변환 방식 (상수)

예시

- `gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)`
- `hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)`

**RGB는 보기용, GRAY는 구조/밝기, HSV는 색 검출**

# 3. OPENCV

---

## ② cv2.inRange() - 색 영역 마스크

`mask = cv2.inRange(src, lowerb, upperb)`  
**특정 값 범위에 속하는 픽셀만 흰색(255) 으로 선택  
나머지는 검정(0)**

주요 파라미터

- `src` : 입력 이미지 → 보통 HSV 이미지
- `lowerb` : 하한값 배열
- `upperb` : 상한값 배열

예시 (파란색)

- `lower_blue = np.array([90, 50, 50])`
- `upper_blue = np.array([130, 255, 255])`
- `mask = cv2.inRange(img_hsv, lower_blue, upper_blue)`

결과 : `mask.shape = (H, W)` 값은 0 또는 255

### 3. OPENCV

---

#### ③ cv2.bitwise\_and() — 마스크 적용

**dst = cv2.bitwise\_and(src1, src2, mask=mask)**

**마스크가 255인 위치만 남김**

**나머지는 0으로 제거**

주요 파라미터

- src1, src2 : 입력 이미지(보통 같은 이미지)
- mask : inRange

예시

result = cv2.bitwise\_and(img\_bgr, img\_bgr, mask=mask)

시각적 의미 : **마스크 통과 영역 → 원래 색 유지**

**탈락 영역 → 검정**



## 3. OPENCV

---

### ④ cv2.split() — 채널 분해

**channels = cv2.split(src)**  
**다채널 이미지를 채널별로 분리**

주요 파라미터

- src : 입력 이미지 (HSV, BGR 등)

예시 (HSV)

- h, s, v = cv2.split(img\_hsv)

결과

h, s, v 모두 (H, W) 형태

각각:h: 색상s: 채도v: 밝기

### 3. OPENCV

---

**실습 예시 코드**

# 3. OPENCV

---

## 실습 1 — 색공간 변환

```
import cv2
import matplotlib.pyplot as plt

# 이미지 읽기
img_bgr = cv2.imread("/content/airplane.jpg")
if img_bgr is None:
    raise FileNotFoundError("이미지 경로를 확인하세요.")

# 색공간 변환
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
```

# 3. OPENCV

RGB



Grayscale



HSV (raw view)



## 실습 1 — 색공간 변환

```
# 시각화
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img_rgb)
plt.title("RGB")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(img_gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(img_hsv)
plt.title("HSV (raw view)")
plt.axis("off")

plt.show()
```

### 3. OPENCV

---

#### 실습 1 — 색공간 변환



#### HSV 결과물이 기괴하고 부자연스러운 색상으로 나타나는 이유

이미지 출력 도구인 matplotlib은  
입력받은 데이터를 기본적으로 RGB 방식으로 해석하여 화면에 뿌려줍니다.  
변환된 `img_hsv` 변수 안에는 RGB 정보가 아닌 완전히 다른 성격의 숫자 값들이  
들어 있습니다.

#### **matplotlib은**

`img_hsv`의 첫 번째 채널 값(H, 색상)을 '빨간색(R)'으로,  
두 번째 채널(S, 채도)을 '초록색(G)'으로,  
세 번째 채널(V, 명도)을 '파란색(B)'으로 강제로 대입해 화면에 출력합니다.

# 3. OPENCV



## 실습 2 — HSV 채널 분해

```
# HSV 채널 분리
h, s, v = cv2.split(img_hsv)

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(h, cmap="gray")
plt.title("Hue")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(s, cmap="gray")
plt.title("Saturation")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(v, cmap="gray")
plt.title("Value (Brightness)")
plt.axis("off")

plt.show()
```

# 3. OPENCV

---

## 실습 3 — HSV로 특정 색 영역 마스크

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 파란색 범위(대략) - 필요시 숫자 조절
lower_blue = np.array([90, 50, 50])
upper_blue = np.array([130, 255, 255])

mask = cv2.inRange(img_hsv, lower_blue, upper_blue)

result = cv2.bitwise_and(img_bgr, img_bgr, mask=mask)
result_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
```

# 3. OPENCV

## 실습 3 — HSV로 특정 색 영역 마스크

### 1단계: 색상 범위 설정 (Targeting)

추출하고 싶은 색상의 **최소치(lower)**와 **최대치(upper)** 범위를 정합니다.

lower\_blue / upper\_blue: HSV 색공간을 기준으로 파란색이 분포하는 영역을 설정.

HSV 사용 이유: 조명이 밝거나 어두워도 파란색이라는 성질(H)은 크게 변하지 않기 때문에, RGB보다 훨씬 안정적으로 색을 찾을 수 있습니다.

### 2단계: 이진화 마스크 생성 (Masking)

범위 안에 들어오는 픽셀만 골라내는 '지도'를 만듭니다.

cv2.inRange(): 이미지의 모든 픽셀을 검사하여 **설정된 파란색 범위에 속하면 흰색(255), 아니면 검은색(0)으로 변환**.

그 결과 mask는 우리가 찾고 싶은 파란색 영역을 알려주는 흑백 지도가 됩니다.

### 3단계: 특정 색상만 남기기 (Filtering)

만든 지도를 원본 이미지에 겹쳐서 파란색 부분만 도려냅니다.

cv2.bitwise\_and(): 원본 이미지(img\_bgr)와 마스크(mask)를 겹쳐서 연산합니다.

마스크가 흰색(1)인 부분 → 원본 색상이 그대로 나타남.

마스크가 검은색(0)인 부분 → 모두 검은색으로 가려짐.

결과: result\_rgb는 배경은 모두 검고 원본의 파란색 물체만 남은 컬러 이미지가 됩니다.



Mask (Blue)



Masked Result (Color)



Original



## 3. OPENCV

### 실습 3 — HSV로 특정 색 영역 마스킹

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(mask, cmap="gray")
plt.title("Mask (Blue)")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(result_rgb)
plt.title("Masked Result (Color)")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.show()
```

# 3. OPENCV

---

## 미니과제 1 — 색공간 비교 리포트 (필수)

과제 내용같은 이미지를 아래 3가지 방식으로 변환해 비교하세요.

- RGB
- Grayscale
- HSV (H, S, v 채널 분리)

제출물 4장의 이미지 출력:

- RGB
- Grayscale
- Hue
- Value

```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
h, s, v = cv2.split(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV))
```

### 3. OPENCV

---

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# 1. 이미지 불러오기
# 본인의 이미지 경로로 변경하세요
image_path = 'sample_image.jpg'
image = cv2.imread(image_path)

# OpenCV는 BGR로 읽으므로 RGB로 변환
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 2. Grayscale 변환
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 3. HSV 변환 및 채널 분리
image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
h_channel, s_channel, v_channel = cv2.split(image_hsv)

# 4. 결과 시각화 (4개 이미지)
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
```

### 3. OPENCV

---

```
# RGB 이미지
axes[0, 0].imshow(image_rgb)
axes[0, 0].set_title('RGB Image', fontsize=14, fontweight='bold')
axes[0, 0].axis('off')

# Grayscale 이미지
axes[0, 1].imshow(image_gray, cmap='gray')
axes[0, 1].set_title('Grayscale', fontsize=14, fontweight='bold')
axes[0, 1].axis('off')

# Hue 채널
axes[1, 0].imshow(h_channel, cmap='hsv')
axes[1, 0].set_title('Hue Channel (H)', fontsize=14, fontweight='bold')
axes[1, 0].axis('off')

# Value 채널
axes[1, 1].imshow(v_channel, cmap='gray')
axes[1, 1].set_title('Value Channel (V)', fontsize=14, fontweight='bold')
axes[1, 1].axis('off')
```

### 3. OPENCV

---

```
plt.tight_layout()
plt.savefig('color_space_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print(" 색공간 비교 완료!")
print(f"- RGB 이미지 shape: {image_rgb.shape}")
print(f"- Grayscale 이미지 shape: {image_gray.shape}")
print(f"- HSV 이미지 shape: {image_hsv.shape}")
```

# 3. OPENCV

---

## 3차시 — 이진화 & 필터링 (노이즈와의 전쟁)



### 차시 목표

- 객체와 배경을 분리하는 감각 획득
- "전처리의 중요성" 체감



### 주요 주제

- Thresholding
- Filtering
- Convolution 개념



### 필수 이론 & 용어

- Global Threshold
- Adaptive Threshold
- Gaussian / Median Filter
- Kernel



### 필수 실습

- 다양한 threshold 비교
- blur 전후 결과 비교

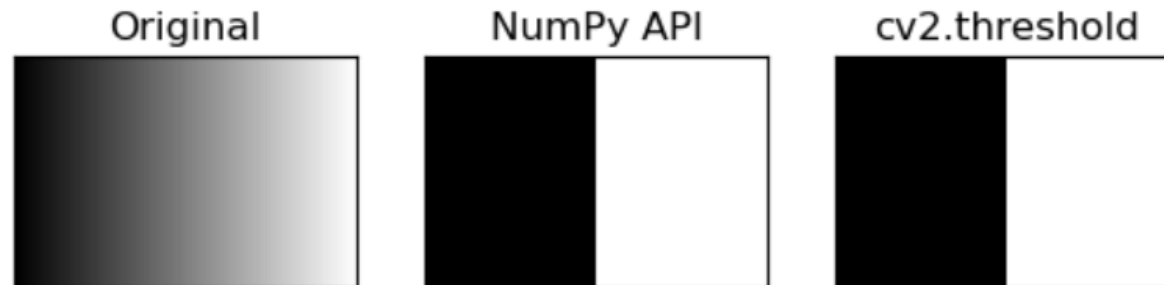
## 3. OPENCV

---

### 1) Thresholding(이진화)

그레이스케일 이미지의 각 픽셀을 임계값(threshold)을 기준으로 0(검은색) 또는 255(흰색)로 변환하는 기법

- 픽셀 값  $\geq$  임계값  $\rightarrow$  255 (흰색, 객체)
- 픽셀 값  $<$  임계값  $\rightarrow$  0 (검은색, 배경)



# 3. OPENCV

---

## 1) Thresholding (이진화)

### (1) Global Threshold

하나의 임계값을 이미지 전체에 적용조명이 균일할 때 효과적  
단점: 그림자/밝기 변화에 취약

### (2) Adaptive Threshold

지역(block)별로 임계값 자동 계산  
조명 불균일 환경에 강함  
계산량 ↑

- Global: 밝기 불균일 → 실패 가능
- Adaptive: 국소 구조 유지 → 객체가 더 잘 살아남음



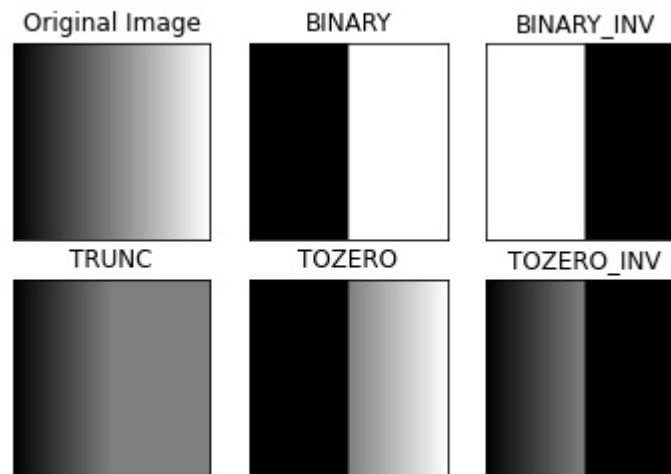
# 3. OPENCV

---

## (1) Global Thresholding

특징: 이미지 전체에 하나의 고정된 임계값 사용

- 빠르고 간단함
- 조명이 균일한 환경에 적합
- 조명 변화에 민감함



# 3. OPENCV

---

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# 이미지 읽기
image = cv2.imread('sample.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 1. Binary Thresholding (기본 이진화)
# 임계값 127: 픽셀 값이 127보다 크면 255, 작으면 0
ret1, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# 2. Binary Inverse (반전 이진화)
ret2, binary_inv = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)

# 3. Truncate (잘라내기)
# 임계값보다 크면 임계값으로 설정, 작으면 그대로
ret3, trunc = cv2.threshold(gray, 127, 255, cv2.THRESH_TRUNC)

# 4. To Zero (임계값 이하 0으로)
ret4, tozero = cv2.threshold(gray, 127, 255, cv2.THRESH_TOZERO)

# 5. Otsu's Method (자동 임계값 계산)
# 이미지 히스토그램을 분석해 최적 임계값 자동 계산
ret5, otsu = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

# 3. OPENCV

---

```
# 결과 시각화
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].imshow(gray, cmap='gray')
axes[0, 0].set_title('Original Grayscale', fontsize=12)
axes[0, 0].axis('off')

axes[0, 1].imshow(binary, cmap='gray')
axes[0, 1].set_title('Binary (thresh=127)', fontsize=12)
axes[0, 1].axis('off')

axes[0, 2].imshow(binary_inv, cmap='gray')
axes[0, 2].set_title('Binary Inverse', fontsize=12)
axes[0, 2].axis('off')

axes[1, 0].imshow(trunc, cmap='gray')
axes[1, 0].set_title('Truncate', fontsize=12)
axes[1, 0].axis('off')

axes[1, 1].imshow(tozero, cmap='gray')
axes[1, 1].set_title('To Zero', fontsize=12)
axes[1, 1].axis('off')

axes[1, 2].imshow(otsu, cmap='gray')
axes[1, 2].set_title(f'Otsu's Method (thresh={ret5:.1f})", fontsize=12)
axes[1, 2].axis('off')

plt.tight_layout()
plt.savefig('global_thresholding.png', dpi=300)
plt.show()

print(f"✅ Otsu's Method - 자동 계산된 임계값: {ret5:.2f}")
```

출처=아시아나 항공



### 3. OPENCV

## 전역 이진화(Global Thresholding)의 대표적인 필터들

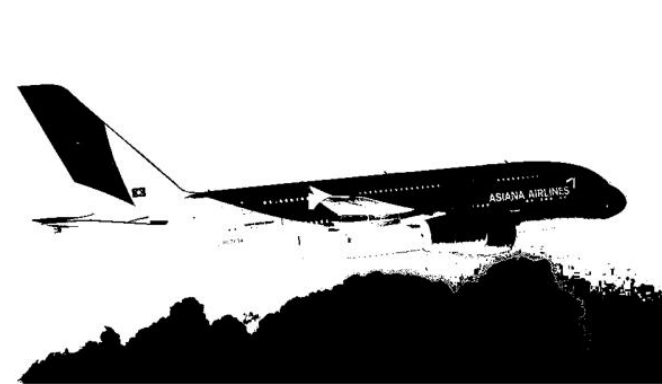
Original Grayscale



Binary (thresh=127)



Binary Inverse



Truncate



To Zero



Otsu's Method (thresh=167.0)





### 3. OPENCV

#### 전역 이진화(Global Thresholding)의 대표적인 필터들

| 타이틀                 | 처리 기법                  | 핵심 개념     | 동작 방식                   | 결과 이미지 특징     | 주 용도     |
|---------------------|------------------------|-----------|-------------------------|---------------|----------|
| Original Grayscale  | Grayscale 변환           | 밝기 정보만 유지 | RGB → 단일 채널(0~255)      | 명암만 존재, 형태 유지 | 전처리 기준   |
| Binary (thresh=127) | Global Threshold (이진화) | 고정 임계값    | 픽셀 > 127 → 255, 그 외 → 0 | 흑/백 명확 분리     | 객체 분리    |
| Binary Inverse      | Inverse Binary         | 이진화 반전    | 픽셀 > 127 → 0, 그 외 → 255 | 배경/객체 반전      | 객체 강조    |
| Truncate            | Truncation Threshold   | 밝기 상한 제한  | 픽셀 > 127 → 127로 고정      | 밝은 영역 평탄화     | 조명 완화    |
| To Zero             | ToZero Threshold       | 조건부 유지    | 픽셀 ≤ 127 → 0, 나머지 유지    | 어두운 영역 제거     | 관심 영역 강조 |
| Otsu's Method       | 자동 이진화                 | 분산 최소화    | 히스토그램 기반<br>임계값 자동 계산   | 데이터 기반 최적 분리  | 산업/자동화   |

출처=아시아나 항공



### 3. OPENCV

#### 전역 이진화(Global Thresholding) 필터의 활용

| 상황          | 추천 기법            |
|-------------|------------------|
| 조명 균일       | Global Threshold |
| 조명 불균일      | Otsu / Adaptive  |
| 밝기 과다       | Truncate         |
| 관심 영역 강조    | ToZero           |
| 객체-배경 반전 필요 | Binary Inverse   |



### 3. OPENCV

---

- **지능형 이진화, Otsu's Method**

히스토그램만 보고 임계값(threshold)을 자동으로 결정하는 전역 이진화 방법  
이미지를 배경(Background)과 객체(Foreground)라는 두 집단으로 나누었을 때,  
각 집단 내의 분산은 최소화하고 두 집단 사이의 분산은 최대화하는 지점을 찾습니다.

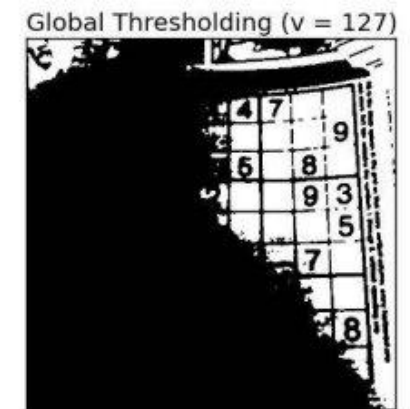
- **알고리즘 단계 (실제 동작)**
  - 그레이스케일 이미지 준비
  - 히스토그램 계산
  - 임계값  $t=0\sim 255$  전부 시도
  - 각  $t$ 에 대해 클래스 분산 계산
  - 분산이 최대가 되는  $t$  선택
  - 그  $t$ 로 이진화

## 3. OPENCV

### (2) Adaptive Thresholding

이미지를 작은 영역(block)으로 나눠 **각 영역마다 다른 임계값 사용**

- 조명 불균일 환경에 강함
- **문서 스캔, OCR에 효과적**
- 계산량이 많음





# 3. OPENCV

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('document.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 1. Global Threshold (비교용)
_, global_thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# 2. Adaptive Mean Threshold
# 주변 픽셀들의 평균값을 임계값으로 사용
adaptive_mean = cv2.adaptiveThreshold(
    gray,                # 입력 이미지
    255,                 # 최대값
    cv2.ADAPTIVE_THRESH_MEAN_C, # 평균값 방식
    cv2.THRESH_BINARY,   # 이진화 타입
    11,                  # 블록 크기 (홀수)
    2                     # 상수 C (평균에서 빼는 값)
)

# 3. Adaptive Gaussian Threshold
# 주변 픽셀들의 가우시안 가중 평균을 임계값으로 사용
adaptive_gaussian = cv2.adaptiveThreshold(
    gray,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C, # 가우시안 가중 평균
    cv2.THRESH_BINARY,
    11,                  # 블록 크기
    2                     # 상수 C
)
```

# 3. OPENCV

---

```
# 결과 비교
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

axes[0, 0].imshow(gray, cmap='gray')
axes[0, 0].set_title('Original Grayscale', fontsize=14, fontweight='bold')
axes[0, 0].axis('off')

axes[0, 1].imshow(global_thresh, cmap='gray')
axes[0, 1].set_title('Global Threshold', fontsize=14, fontweight='bold')
axes[0, 1].axis('off')

axes[1, 0].imshow(adaptive_mean, cmap='gray')
axes[1, 0].set_title('Adaptive Mean', fontsize=14, fontweight='bold')
axes[1, 0].axis('off')

axes[1, 1].imshow(adaptive_gaussian, cmap='gray')
axes[1, 1].set_title('Adaptive Gaussian', fontsize=14, fontweight='bold')
axes[1, 1].axis('off')

plt.tight_layout()
plt.savefig('adaptive_thresholding.png', dpi=300)
plt.show()
```

출처=아시아나 항공

### 3. OPENCV



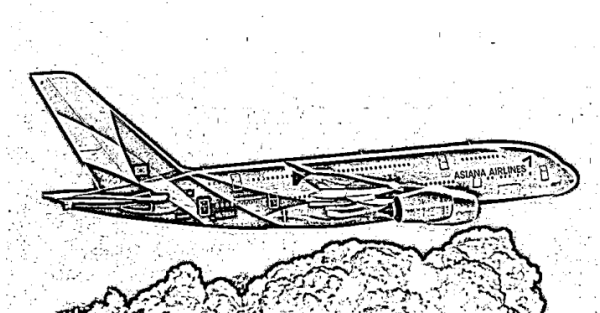
Original Grayscale



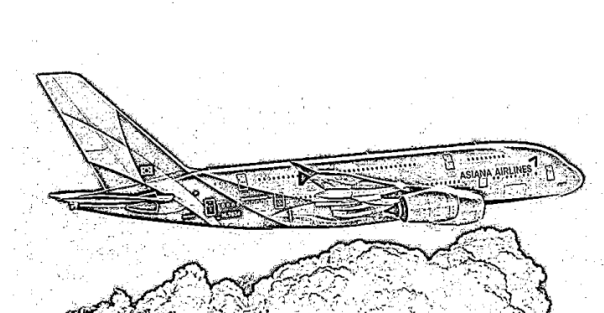
Global Threshold



Adaptive Mean



Adaptive Gaussian)



# 3. OPENCV

---

## 3) Filtering (필터링)

Filtering : 이미지의 각 픽셀을 주변 픽셀들과의 연산을 통해 새로운 값으로 변환하는 기법

- 이진화 전 노이즈 제거 → 깨끗한 이진화 결과
- 객체 경계 선명화 → 정확한 객체 검출
- 불필요한 디테일 제거 → 핵심 패턴 강조

### Gaussian Filter

가우시안 분포 기반 부드러운 블러  
노이즈 완화 + 경계 유지(상대적으로)

### Median Filter

주변 픽셀의 중앙값  
소금-후추(salt & pepper)  
노이즈 제거에 강함

## 3. OPENCV

### Gaussian Filter (가우시안 필터)

가우시안(정규분포) 함수를 이용해 중심 픽셀에 가까울수록 큰 가중치를 부여

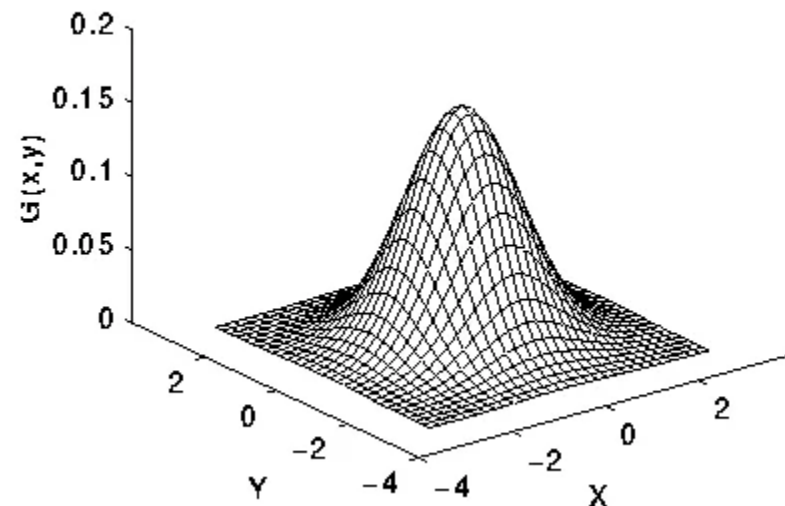
특징: 자연스러운 블러 효과  
노이즈 완화하면서 경계 어느 정도 유지  
이미지 부드럽게 만들기  
고주파 노이즈 제거

$$\frac{1}{16} \times$$

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

$$\frac{1}{273} \times$$

|   |    |    |    |   |
|---|----|----|----|---|
| 1 | 4  | 7  | 4  | 1 |
| 4 | 16 | 26 | 16 | 4 |
| 7 | 26 | 41 | 26 | 7 |
| 4 | 16 | 26 | 16 | 4 |
| 1 | 4  | 7  | 4  | 1 |



# 3. OPENCV

---

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# 이미지 읽기
image = cv2.imread('sample.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 가우시안 노이즈 추가 (시뮬레이션)
noise = np.random.normal(0, 25, image_rgb.shape).astype(np.uint8)
noisy_image = cv2.add(image_rgb, noise)

# Gaussian Filter 적용 (다양한 커널 크기)
# 커널 크기가 클수록 더 강한 블러
gaussian_3x3 = cv2.GaussianBlur(noisy_image, (3, 3), 0)
gaussian_5x5 = cv2.GaussianBlur(noisy_image, (5, 5), 0)
gaussian_9x9 = cv2.GaussianBlur(noisy_image, (9, 9), 0)

# 시그마 값 조정 (표준편차)
gaussian_sigma1 = cv2.GaussianBlur(noisy_image, (5, 5), sigmaX=1)
gaussian_sigma3 = cv2.GaussianBlur(noisy_image, (5, 5), sigmaX=3)
```

# 3. OPENCV

---

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].imshow(image_rgb)
axes[0, 0].set_title('original', fontsize=13, fontweight='bold')
axes[0, 0].axis('off')

axes[0, 1].imshow(noisy_image)
axes[0, 1].set_title('noise', fontsize=13, fontweight='bold')
axes[0, 1].axis('off')

axes[0, 2].imshow(gaussian_3x3)
axes[0, 2].set_title('Gaussian 3×3\n(weak blur)', fontsize=13, fontweight='bold')
axes[0, 2].axis('off')

axes[1, 0].imshow(gaussian_5x5)
axes[1, 0].set_title('Gaussian 5×5\n(median blur)', fontsize=13, fontweight='bold')
axes[1, 0].axis('off')

axes[1, 1].imshow(gaussian_9x9)
axes[1, 1].set_title('Gaussian 9×9\n(strong blur)', fontsize=13, fontweight='bold')
axes[1, 1].axis('off')

axes[1, 2].imshow(gaussian_sigma3)
axes[1, 2].set_title('Gaussian  $\sigma=3$ \n(high std)', fontsize=13, fontweight='bold')
axes[1, 2].axis('off')

plt.tight_layout()
plt.savefig('gaussian_filtering.png', dpi=300)
plt.show()
```

# 3. OPENCV

## 커널이 클수록 더 부드러운 이미지

커널 크기  $\uparrow \rightarrow$  더 많은 픽셀 평균화  $\rightarrow$  표준편차  $\downarrow \rightarrow$  부드러운 이미지

"부드럽다" = 픽셀 간 차이가 작다 = 표준편차가 작다

original



noise



Gaussian 3x3  
(weak blur)



Gaussian 5x5  
(median blur)



Gaussian 9x9  
(strong blur)



Gaussian  $\sigma=3$   
(high std)





# 3. OPENCV

---

## Salt & Pepper Noise

이미지의 무작위 픽셀이 갑자기 최댓값(255, 흰색) 또는 최솟값(0, 검은색)으로 변하는 노이즈

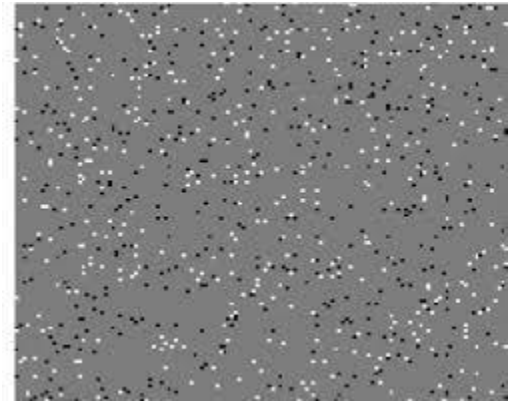
- Salt (소금) → 흰색 점 (255)
- Pepper (후추) → 검은색 점 (0)

### 1. 발생 원인

- 센서 결함: 카메라 센서의 dead pixel
- 전송 에러: 데이터 전송 중 비트 오류
- 아날로그-디지털 변환 오류: ADC 변환 과정의 문제
- 먼지/이물질: 렌즈나 센서 표면의 오염

### 2. 특성

- 극단값 노이즈: 0 또는 255만 발생
- 무작위 위치: 예측 불가능한 위치에 발생
- 독립적: 주변 픽셀과 무관
- 희소성: 전체 픽셀의 일부만 영향



# 3. OPENCV

---

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Read image
image = cv2.imread('asiana.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Add Salt & Pepper noise
def add_salt_pepper_noise(image, salt_prob=0.02, pepper_prob=0.02):
    noisy = image.copy()

    # Add Salt (white dots)
    num_salt = np.ceil(salt_prob * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_salt))
               for i in image.shape[:2]]
    noisy[coords[0], coords[1], :] = 255

    # Add Pepper (black dots)
    num_pepper = np.ceil(pepper_prob * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_pepper))
               for i in image.shape[:2]]
    noisy[coords[0], coords[1], :] = 0

    return noisy
```

# 3. OPENCV

---

```
noisy_sp = add_salt_pepper_noise(image_rgb)

# Apply Median Filter (various kernel sizes)
median_3 = cv2.medianBlur(noisy_sp, 3)
median_5 = cv2.medianBlur(noisy_sp, 5)
median_9 = cv2.medianBlur(noisy_sp, 9)

# Compare with Gaussian (on same noise)
gaussian_compare = cv2.GaussianBlur(noisy_sp, (5, 5), 0)
```

# 3. OPENCV

---

```
# Visualize results
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].imshow(image_rgb)
axes[0, 0].set_title('Original Image', fontsize=13, fontweight='bold')
axes[0, 0].axis('off')

axes[0, 1].imshow(noisy_sp)
axes[0, 1].set_title('Salt & Pepper Noise\n(Impulse Noise)', fontsize=13, fontweight='bold')
axes[0, 1].axis('off')

axes[0, 2].imshow(median_3)
axes[0, 2].set_title('✅ Median Filter 3×3\n(Noise Removed)', fontsize=13, fontweight='bold', color='green')
axes[0, 2].axis('off')

axes[1, 0].imshow(median_5)
axes[1, 0].set_title('✅ Median Filter 5×5\n(Cleaner)', fontsize=13, fontweight='bold', color='green')
axes[1, 0].axis('off')

axes[1, 1].imshow(median_9)
axes[1, 1].set_title('Median Filter 9×9\n(Excessive Blur)', fontsize=13, fontweight='bold')
axes[1, 1].axis('off')

axes[1, 2].imshow(gaussian_compare)
axes[1, 2].set_title('❌ Gaussian 5×5\n(Noise Smearing)', fontsize=13, fontweight='bold', color='red')
axes[1, 2].axis('off')

plt.tight_layout()
plt.savefig('median_filtering.png', dpi=300)
plt.show()
```

# 3. OPENCV

---

**Original Image**



**Salt & Pepper Noise  
(Impulse Noise)**



**Median Filter 3×3  
(Noise Removed)**



**Median Filter 5×5  
(Cleaner)**



**Median Filter 9×9  
(Excessive Blur)**



**Gaussian 5×5  
(Noise Smearing)**



### 3. OPENCV



Salt & Pepper 노이즈 = 흰색(255) 또는 검은색(0) 점들이 무작위로 튀어나옴

- **Median Filter** ✓ → 극단값 무시 → 노이즈 완벽 제거  
극단값(255)을 완전히 무시하고 정상적인 값(121)으로 복원!

```
# 예시: 3x3 영역에서 중앙 픽셀이 Salt 노이즈
주변 9개 픽셀: [120, 122, 255, 118, 121, 119, 123, 120, 124]
               ↓ 정렬
정렬된 값:     [118, 119, 120, 120, 121, 122, 123, 124, 255]
               ↑
               중앙값 = 121
```

- **Gaussian Filter** ✗ → 극단값 평균화 → 노이즈가 번짐  
극단값(255)이 평균을 끌어올려 주변까지 밝아짐!  
정규분포 평균을 중심으로 높은 가중치, 좌우끝단으로 갈수록 작아지는 가중치분포

```
# 같은 3x3 영역
주변 9개 픽셀: [120, 122, 255, 118, 121, 119, 123, 120, 124]
               ↓ 가중 평균
가우시안 가중치: [1, 2, 1, 2, 4, 2, 1, 2, 1] / 16
               ↓
결과 = (120×1 + 122×2 + 255×1 + ... + 124×1) / 16
      ≈ 135 (원래는 121이어야 함)
```

# 3. OPENCV

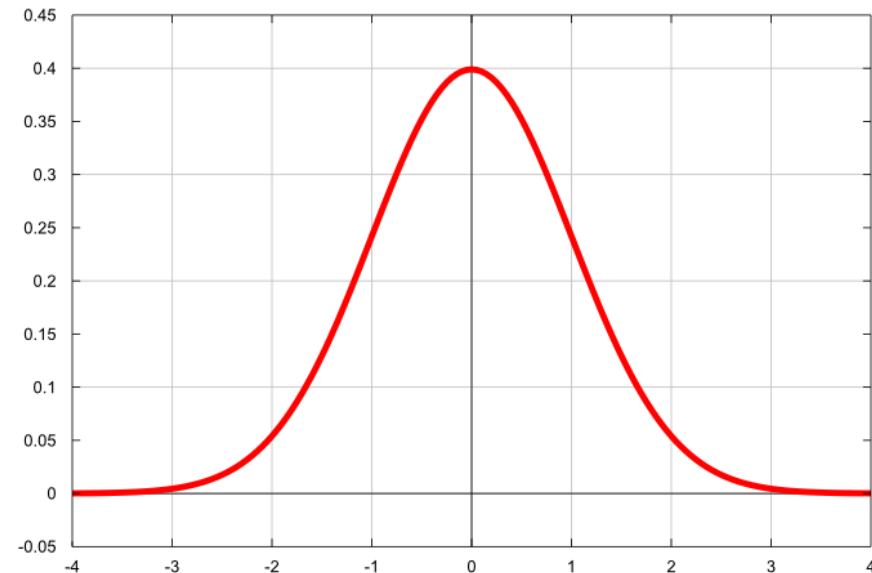
---

## 언제 Median ?

- Salt & Pepper 노이즈 (흰색/검은색 점)
- 센서 결함 (dead pixel)
- 전송 에러로 인한 픽셀 손상
- 스캔 문서의 먼지/얼룩

## 언제 Gaussian?

- 연속적인 노이즈 (카메라 열 노이즈)
- 전체적으로 거친 이미지
- 부드럽게 만들고 싶을 때
- Salt & Pepper에는 절대 사용 금지!



## 3. OPENCV

### 4) Convolution 개념

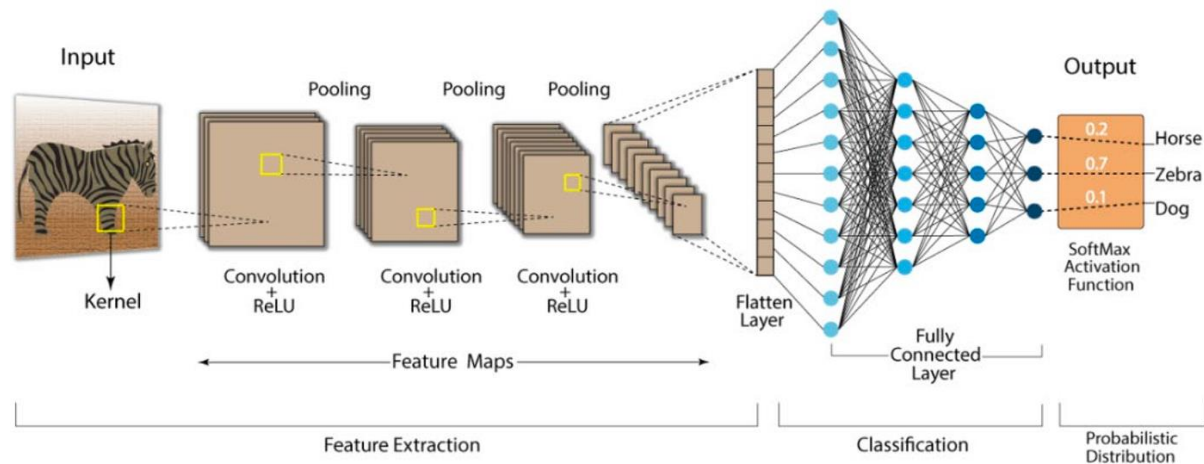
필터링의 수학적 본질

커널(작은 행렬)을 이미지 위에서 슬라이딩하며 각 위치에서 가중합(weighted sum)을 계산하는 연산

#### ◆ Kernel

필터의 "모양과 크기"

예:  $3 \times 3$ ,  $5 \times 5$  커널수록 → 더 부드러움 / 디테일 손실





# 3. OPENCV

---

## Matplot에서 한글 사용시 다음코드를 포함 !

```
import cv2
import numpy as np
from PIL import ImageFont, ImageDraw, Image

def put_korean_text(img, text, position, font_path, font_size, color):
    # OpenCV 이미지를 PIL 이미지로 변환
    img_pil = Image.fromarray(img)
    draw = ImageDraw.Draw(img_pil)

    # 폰트 로드 (Windows는 "batang.ttc", "malgun.ttf" 등 경로 지정)
    font = ImageFont.truetype(font_path, font_size)

    # 텍스트 그리기
    draw.text(position, text, font=font, fill=color)

    # 다시 OpenCV(numpy) 배열로 변환
    return np.array(img_pil)

# 실전 사용 예시
img = cv2.imread('input.jpg')
img = put_korean_text(img, "특징점 검출 완료", (10, 30), "malgun.ttf", 20, (255, 255, 255))
cv2.imshow('Result', img)
```

# 3. OPENCV

# 다양한 커널 정의

```
kernels = {  
    'Identity\n(원본 유지)': np.array([  
        [0, 0, 0],  
        [0, 1, 0],  
        [0, 0, 0]  
    ], dtype=np.float32),  
  
    'Box Blur\n(평균 블러)': np.ones((5, 5), dtype=np.float32) / 25,  
  
    'Gaussian-like\n(가우시안 근사)': np.array([  
        [1, 2, 1],  
        [2, 4, 2],  
        [1, 2, 1]  
    ], dtype=np.float32) / 16,  
  
    'Sharpen\n(선명화)': np.array([  
        [0, -1, 0],  
        [-1, 5, -1],  
        [0, -1, 0]  
    ], dtype=np.float32),  
  
    'Edge Detection\n(수직 엣지)': np.array([  
        [-1, 0, 1],  
        [-2, 0, 2],  
        [-1, 0, 1]  
    ], dtype=np.float32), # Sobel X
```

```
'Edge Detection\n(수평 엣지)': np.array([  
    [-1, -2, -1],  
    [0, 0, 0],  
    [1, 2, 1]  
], dtype=np.float32), # Sobel Y
```

```
'Emboss\n(양각 효과)': np.array([  
    [-2, -1, 0],  
    [-1, 1, 1],  
    [0, 1, 2]  
], dtype=np.float32),
```

```
'Laplacian\n(엣지 검출)': np.array([  
    [0, 1, 0],  
    [1, -4, 1],  
    [0, 1, 0]  
], dtype=np.float32)  
}
```

# 3. OPENCV

## 필터별(커널별) 특징

| 커널 이름               | 합(sum) | 중심값  | 핵심 원리                                            | 결과적으로 강조되는 것               | 왜 잘 되는가? (직관적 설명)                                        |
|---------------------|--------|------|--------------------------------------------------|----------------------------|----------------------------------------------------------|
| Identity            | 1.0    | 1.0  | 주변 영향 0, 중심만 그대로 복사                              | 원본 그대로                     | 변화 없음 → 필터링 테스트용 기준 이미지                                  |
| Box Blur (5×5)      | 1.0    | —    | 모든 이웃에 <b>동일한 가중치</b><br>→ 국부 평균                 | 저주파(큰 패턴, 색상 변화 완만한 영역)    | 고주파(날카로운 경계) 성분을 평균화<br>→ 노이즈 제거, 부드러운 블러 효과             |
| Gaussian-like (3×3) | 1.0    | 0.25 | 중심에 무게 많이, 멀어질수록 급격히 감소 (가우시안 분포 근사)             | 저주파 + 자연스러운 블러             | Box보다 자연스럽게,<br>가장자리 아티팩트 적음<br>(현실적 블러에 가장 많이 씬)        |
| Sharpen             | 1.0    | 5.0  | 중심 매우 크게 강조<br>+ 주변 <b>음의 값</b> 으로 빼기            | 고주파(엣지, 디테일) 강조            | 원본 + (원본 - 블러) 형태<br>→ unsharp masking 원리, 날카로움 ↑        |
| Sobel X (수직 엣지)     | 0.0    | 0.0  | 좌우 밝기 차이 계산<br>(미분 근사)                           | <b>수직 방향</b> 밝기 급변 (수직 엣지) | 왼쪽 음수, 오른쪽 양수 → 좌→우 밝기 증가<br>영역 강조, 수평 방향 변화 억제          |
| Sobel Y (수평 엣지)     | 0.0    | 0.0  | 위아래 밝기 차이 계산                                     | <b>수평 방향</b> 밝기 급변 (수평 엣지) | 위쪽 음수, 아래쪽 양수<br>→ 위→아래 밝기 증가 영역 강조                      |
| Emboss              | 1.0    | 1.0  | 대각선 방향으로 <b>음수 ↔ 양수</b><br>극단적 차이 + 중심 강조        | 3D 양각/음각 같은 조명 효과          | 한 방향은 음수, 반대 방향은 양수<br>→ 그림자/하이라이트처럼 보이게 만들              |
| Laplacian           | 0.0    | -4.0 | 2차 미분 근사<br>→ 밝기 변화의 <b>변화율의 변화</b><br>(엣지에서 극값) | 모든 방향의 <b>엣지</b> (2차 미분)   | 엣지에서 0을 지나 양<br>→ 음 또는 음→양으로 급변<br>→ <b>엣지 위치 정확히 잡음</b> |

# 3. OPENCV

---

## 커널별 3가지 패턴 요약

### 저주파 강조 (Blur 계열)

- 커널 합 = 1, 모든 값 양수 또는 중심에 무게 집중
- 고주파(날카로운 변화)를 평균화해서 제거

### 고주파 강조 (Sharpen, Laplacian)

- 중심값이 매우 크거나 음수
- 주변과의 차이를 극대화
- 엣지, 디테일, 잡음까지 함께 강조됨

### 방향성 엣지 검출 (Sobel, Prewitt 계열)

- 합 = 0, 중심 0 또는 작음
- 한쪽은 양수, 반대쪽은 음수
- 특정 방향의 1차 미분 근사
- 방향에 따라 다른 엣지만 살아남

# 3. OPENCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import platform

# 운영체제 별 폰트 설정
if platform.system() == 'Windows':
    plt.rc('font', family='Malgun Gothic')
elif platform.system() == 'Darwin': # Mac
    plt.rc('font', family='AppleGothic')
else: # Linux/Colab (나눔 폰트 설치 필요)
    plt.rc('font', family='NanumGothic')

# 마이너스 기호 깨짐 방지
plt.rcParams['axes.unicode_minus'] = False

# 위에 주어진 kernels 딕셔너리 그대로 사용
# (kernels = { ... } 코드 복사해서 붙여넣기)
```

```
# 다양한 커널 정의
kernels = {
    'Identity\n(원본 유지)': np.array([
        [0, 0, 0],
        [0, 1, 0],
        [0, 0, 0]
    ], dtype=np.float32),
    'Box Blur\n(평균 블러)': np.ones((5, 5), dtype=np.float32) / 25,
    'Gaussian-like\n(가우시안 근사)': np.array([
        [1, 2, 1],
        [2, 4, 2],
        [1, 2, 1]
```

### 3. OPENCV

---

```
img = cv2.imread('asiana.jpg', cv2.IMREAD_GRAYSCALE) # 그레이스케일 추천
if img is None:
    img = np.random.randint(0, 255, (400, 600), np.uint8) # 테스트용 랜덤 이미지

results = {}
for name, kernel in kernels.items():
    # 컨볼루션 (경계 처리: reflect로 아티팩트 줄임)
    filtered = cv2.filter2D(img, cv2.CV_32F, kernel)
    # 시각화를 위해 0~255 범위로 정규화
    filtered = cv2.normalize(filtered, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    results[name] = filtered

# 시각화
plt.figure(figsize=(15, 10))
for i, (name, res) in enumerate(results.items(), 1):
    plt.subplot(3, 3, i)
    plt.imshow(res, cmap='gray')
    plt.title(name.replace('\n', ' '), fontsize=9)
    plt.axis('off')

plt.tight_layout()
plt.show()
```

# 3. OPENCV

---

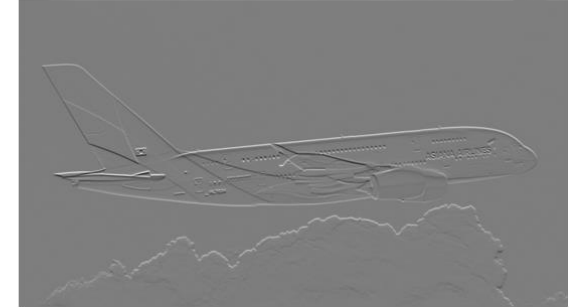
원본 이미지 (Original Image)



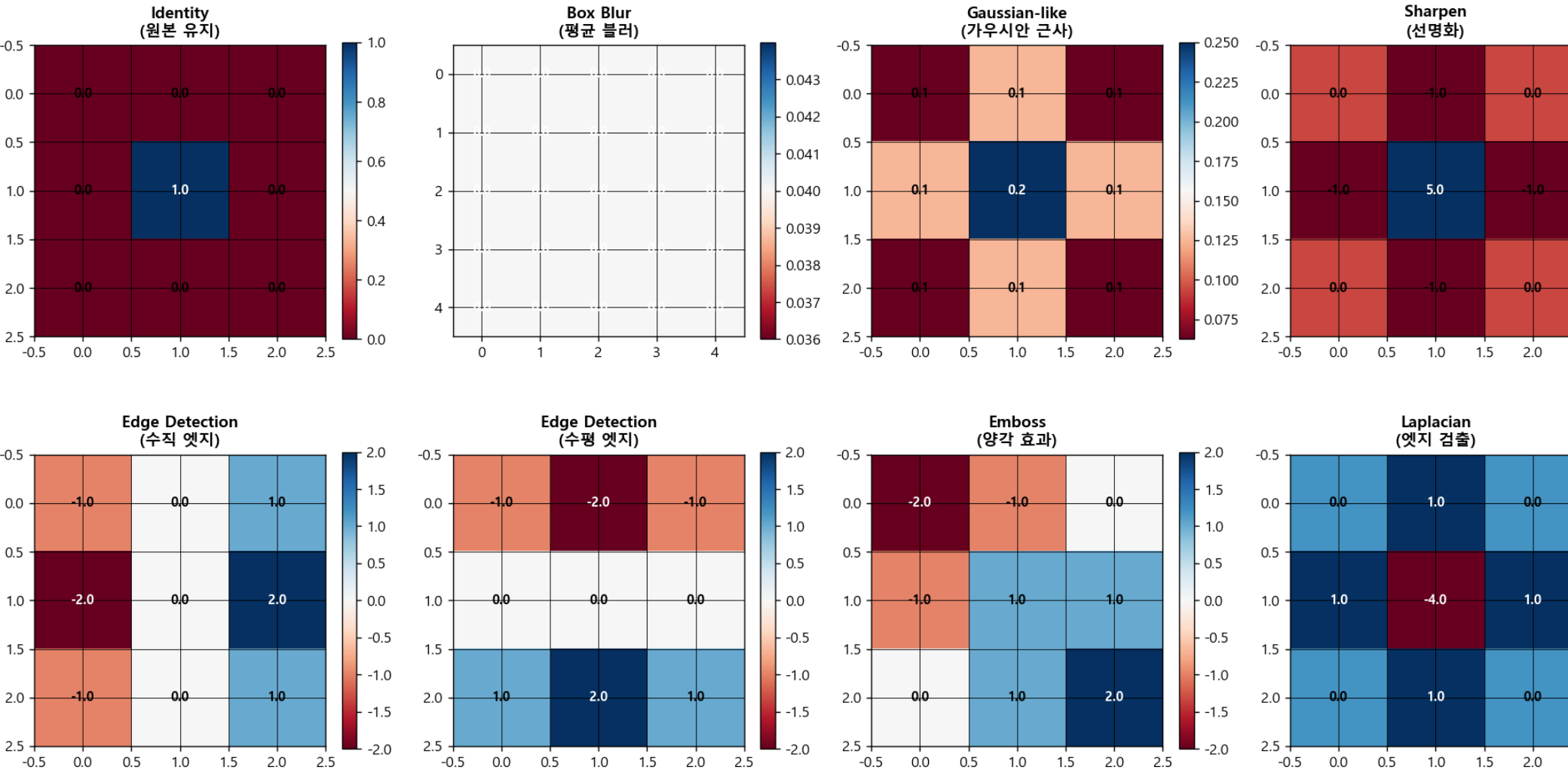
Edge Detection (수직 엣지)



Edge Detection (수평 엣지)



# 3. OPENCV





# 3. OPENCV

원본 이미지



Gaussian-like  
(가우시안 근사)



Edge Detection  
(수평 엣지)



(원본 유지)



Sharpen  
(선명화)



Emboss  
(양각 효과)



(평균 블러)



Edge Detection  
(수직 엣지)



### **3. OPENCV**

---

## **주요 명령어 파라미터**

## 3. OPENCV

---

### ① `cv2.threshold()` — Global Threshold (전역 이진화)

**`retval, dst = cv2.threshold(src, thresh, maxval, type)`**

주요 파라미터

- `src` : 입력 이미지 (Grayscale 필수)
- `thresh` : 임계값 `maxval` : 조건 만족 시 부여할 값 (보통 255)
- `type` : 이진화 방식 자주 쓰는 type
  - `cv2.THRESH_BINARY`
  - `cv2.THRESH_BINARY_INV`

코드에서의 사용

```
_, th_global = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

내부 동작 `if pixel > 127 → 255 else → 0`

간단 예시 (INV) `_, th_inv = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY_INV)`

**주의점 : 조명 변화에 매우 취약, 그림자 많은 이미지에서 실패 잦음**

## 3. OPENCV

---

### ② cv2.adaptiveThreshold() — Adaptive Threshold (지역 이진화)

**dst = cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)**

주요 파라미터

- src : 입력 이미지 (Grayscale 필수)
- maxValue : 결과 흰색 값 (보통 255)
- adaptiveMethod : cv2.ADAPTIVE\_THRESH\_MEAN\_C
  - cv2.ADAPTIVE\_THRESH\_GAUSSIAN\_C
- thresholdType : cv2.THRESH\_BINARY
  - cv2.THRESH\_BINARY\_INV
- blockSize : 주변 영역 크기 (홀수만 가능)
- C : 계산된 임계값에서 빼는 보정값

### 3. OPENCV

---

“내 주변 평균보다 조금 밝으면 객체”

```
th_adapt = cv2.adaptiveThreshold(  
    gray, 255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    blockSize=11,  
    C=2  
)
```

(MEAN 방식)

```
th_mean = cv2.adaptiveThreshold(  
    gray, 255,  
    cv2.ADAPTIVE_THRESH_MEAN_C,  
    cv2.THRESH_BINARY,  
    15, 5  
)
```

## 3. OPENCV

---

### ③ cv2.GaussianBlur() — 가우시안 블러

**dst = cv2.GaussianBlur(src, ksize, sigmaX)**  
**가우시안 분포 기반 블러, 노이즈 완화 → 이진화 안정화**

주요 파라미터

- src : 입력 이미지
- ksize : 커널 크기 (홀수, 홀수)
- sigmaX : X축 표준편차 0 → OpenCV가 자동 계산

코드에서의 사용

- blur = cv2.GaussianBlur(gray, (5,5), 0)

작은 노이즈 제거  
경계는 비교적 유지

### 3. OPENCV

---

**실습 예시 코드**

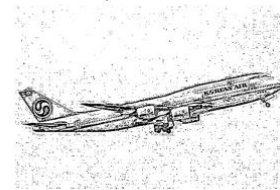
Grayscale



Global Threshold



Adaptive Threshold



### 3. OPENCV

---

#### 실습 1 — Threshold 비교 : Global vs Adaptive

```
import cv2
import matplotlib.pyplot as plt

# 이미지 읽기 & 그레이 변환
img = cv2.imread("/content/airplane.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Global Threshold
_, th_global = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Adaptive Threshold
th_adapt = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=11,
    C=2
)
```



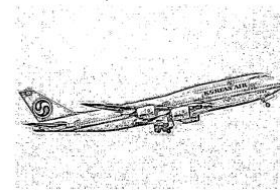
Grayscale



Global Threshold



Adaptive Threshold



### 3. OPENCV

---

#### 실습 1 — Threshold 비교 : Global vs Adaptive

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(th_global, cmap="gray")
plt.title("Global Threshold")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(th_adapt, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.show()
```

Threshold (No Blur)

Threshold (Gaussian Blur)

Blurred Image



### 3. OPENCV

---

#### 실습 2 — 블러 전/후 이진화 비교

블러 없음 → 노이즈까지 객체로 인식  
블러 적용 → 객체 윤곽만 남음

```
# Gaussian Blur
blur = cv2.GaussianBlur(gray, (5,5), 0)

_, th_no_blur = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
_, th_blur = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY)
```

Threshold (No Blur)



Threshold (Gaussian Blur)



Blurred Image



# 3. OPENCV

## 실습 2 — 블러 전/후 이진화 비교

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(th_no_blur, cmap="gray")
plt.title("Threshold (No Blur)")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(th_blur, cmap="gray")
plt.title("Threshold (Gaussian Blur)")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(blur, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")

plt.show()
```

# 3. OPENCV

---

## 원본 이미지

- (Filtering) 노이즈 제거
- (Thresholding) 객체/배경 분리
- (Contour / Edge / Detection)

**“이제 객체와 배경을 나눌 수 있다.**

**그렇다면,이 경계는 어디에 있는가?”**

- 3차시: Edge & Contour (형태를 잡다)

# 3. OPENCV

---

## 2. 미니 과제: "노이즈 극복! 특정 색상 영역 검출기"

영상 내의 노이즈를 제거하고,  
특정 색상(예: 파란색)만 정확하게 추출하여 이진화

1. 카메라 영상(또는 파일)을 읽어와 실시간으로 화면에 표시합니다.
2. 영상에 섞인 지저분한 노이즈를 가우시안 필터를 사용해 제거합니다.
3. 인간의 인식에 친화적인 HSV 색공간으로 변환하여 특정 색 영역을 설정합니다.
4. 검출된 영역을 이진화하여 마스크 이미지를 만들고 결과 창에 띄웁니다.

# 3. OPENCV

---

```
import cv2
import numpy as np

# 1. 영상 입력 설정 (파일 또는 카메라)
cap = cv2.VideoCapture(0)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break

    # 2. 필터링: 가우시안 블러로 노이즈 제거 (노이즈와의 전쟁)
    # Gaussian-like 커널 원리를 적용하여 영상을 부드럽게 만듦
    blur = cv2.GaussianBlur(frame, (5, 5), 0)

    # 3. 색공간 변환: BGR -> HSV (색/밝기 분리)
    hsv = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)

    # 4. 특정 색상 범위 설정 (예: 파란색 영역)
    lower_blue = np.array([100, 100, 100])
    upper_blue = np.array([130, 255, 255])
```

### 3. OPENCV

---

# 5. 이진화: 범위 내 픽셀은 흰색(255), 나머지는 검은색(0) 처리

```
mask = cv2.inRange(hsv, lower_blue, upper_blue)
```

# 6. 결과 시각화 (Drawing)

```
cv2.imshow('Original Video', frame)
```

```
cv2.imshow('Noise Reduced (Blur)', blur)
```

```
cv2.imshow('Blue Color Detect (Binary)', mask)
```

# 'q' 키를 누르면 종료

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

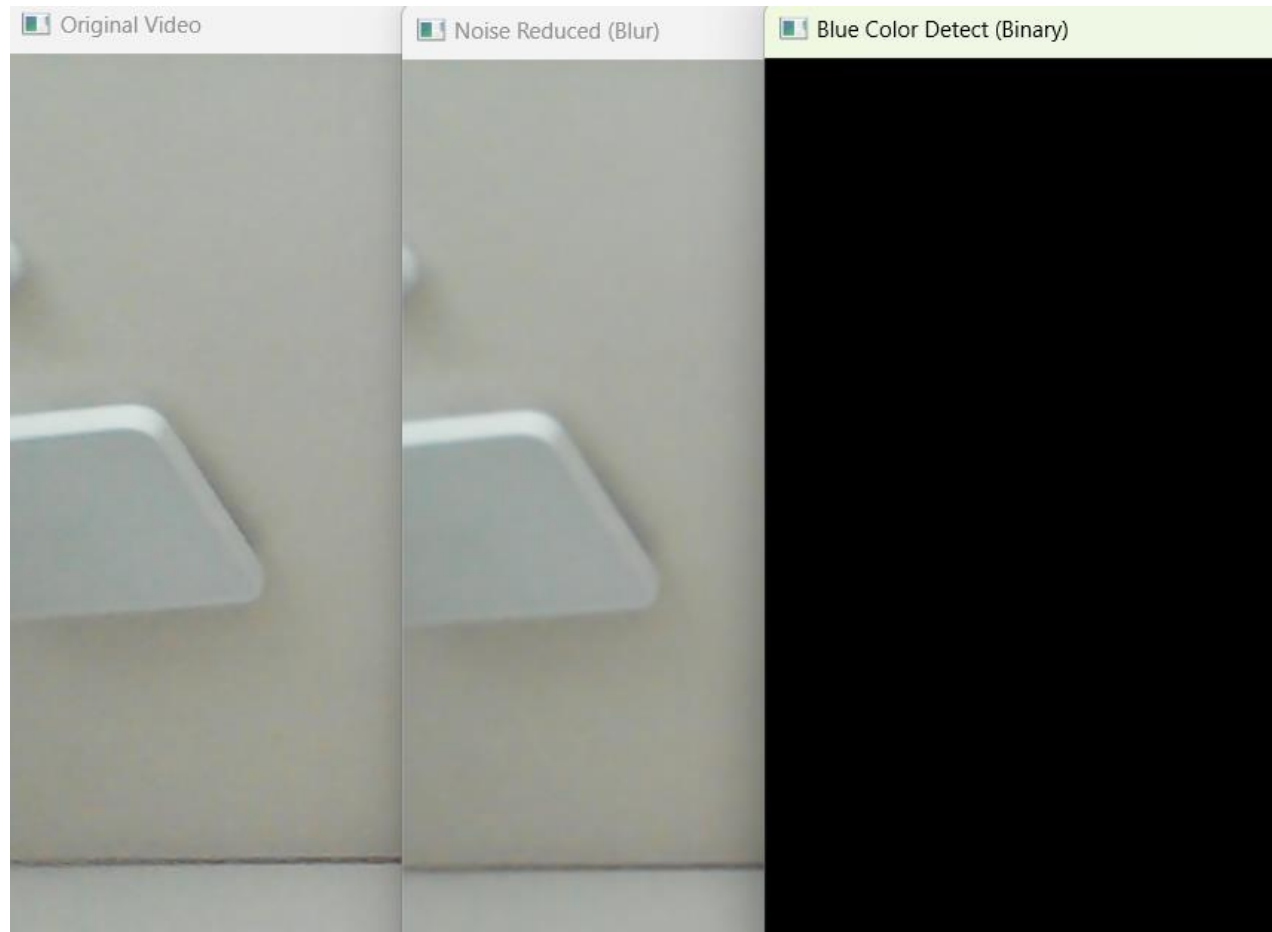
```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

# 3. OPENCV

---





### 3. OPENCV

---

```
import cv2
import numpy as np
import os

# 1. 파일 로드 및 경로 확인
file_path = 'asiana.jpg'
if not os.path.exists(file_path):
    print(f"에러: '{file_path}' 파일을 찾을 수 없습니다.")
else:
    # 이미지 읽기
    img = cv2.imread(file_path)
    h, w = img.shape[:2] # 이미지 크기 정보 획득

# 2. 필터링: 가우시안 블러 (노이즈와의 전쟁)
# Gaussian-like 커널(주변부 가중치 분산)을 활용해 노이즈 제거
blur = cv2.GaussianBlur(img, (5, 5), 0)

# 3. 색공간 변환: BGR -> HSV (색/밝기 분리)
# 특정 색상을 검출하기 위해 인식 친화적인 HSV 좌표계 사용
hsv = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)
```

### 3. OPENCV

---

# 4. 이진화 (Thresholding): 특정 색 영역 추출

# 예: 파란색 영역 (Lower/Upper 범위 설정)

```
lower_blue = np.array([100, 100, 100])
```

```
upper_blue = np.array([130, 255, 255])
```

```
mask = cv2.inRange(hsv, lower_blue, upper_blue)
```

# 5. 결과 시각화 및 파일 저장

# 원본 이미지 위에 텍스트 그리기 (Drawing)

```
cv2.putText(img, "Original", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

# 수정본 로컬 저장

```
cv2.imwrite('processed_mask.jpg', mask)
```

```
print("성공: 결과 이미지가 'processed_mask.jpg'로 저장되었습니다.")
```

# 결과 화면 출력

```
cv2.imshow('1. Original', img)
```

```
cv2.imshow('2. Gaussian Blur', blur)
```

```
cv2.imshow('3. Binary Mask', mask)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

### 3. OPENCV

---

#### H: 100~130

- 청록(cyan) ~ 순수 파랑 ~ 보라빛이 조금 섞인 파랑까지 포함
- 대부분의 "파란색" 물체(하늘, 옷, 표지판, 차량 등)를 잘 잡음
- 너무 넓히면 (예: 90~150) 초록/보라가 섞일 수 있음

#### S: 100~255

- 채도가 어느 정도 있는 색만 잡음 ( $s < 100$ 이면 회색/흰색/연한 파랑이 포함됨)
- 흐릿하거나 밝은 회색빛 파랑은 제외
- 더 선명한 블루에 집중

#### V: 100~255

- 어두운 파랑(거의 검정에 가까운)은 제외
- 밝고 선명한 파랑 위주로 검출 (조명 좋은 실내/야외에서 유리)

이 범위는 **밝고 채도가 높은 선명한 파란색을 탐지**하는 데 아주 적합합니다.  
많은 실시간 객체 추적/색상 기반 마스킹 프로젝트에서 비슷한 값을 씁니다.

# 3. OPENCV

